IEEE ComSoc IEEE Communications Society | IEEE COMPUTER SOCIETY | IEEE Signal Processing Society | VTS Connecting the Mobile World | IEEE Transactions on Machine Learning in Communications and Networking

Received 13 September 2024; revised 23 December 2024; accepted 4 January 2025.
Date of publication 9 January 2025; date of current version 17 January 2025.

The associate editor coordinating the review of this article and approving it for publication was C. Shen.

Digital Object Identifier 10.1109/TMLCN.2025.3527919

Asynchronous Real-Time Federated Learning for Anomaly Detection in Microservice Cloud Applications

MAHSA RAEISZADEH[1], AMIN EBRAHIMZADEH[1], ROCH H. GLITHO[1] (Life Senior Member, IEEE), JOHAN EKER[2], AND RAQUEL A. F. MINI[2]

[1]CIISE, Concordia University, Montreal, QC H3G 1M8, Canada
[2]Ericsson Research, 223 62 Lund, Sweden

CORRESPONDING AUTHOR: M. RAEISZADEH (mahsa.raeiszadeh@concordia.ca)

This work was supported by the Ericsson/Evolution of Networked Services through a Corridor in Québec and Ontario for Research and Innovation (ENCQOR-5G) Industrial Research Chair on Cloud and Edge Computing for 5G and Beyond.

ABSTRACT The complexity and dynamicity of microservice architectures in cloud environments present substantial challenges to the reliability and availability of the services built on these architectures. Therefore, effective anomaly detection is crucial to prevent impending failures and resolve them promptly. Distributed data analysis techniques based on machine learning (ML) have recently gained attention in detecting anomalies in microservice systems. ML-based anomaly detection techniques mostly require centralized data collection and processing, which may raise scalability and computational issues in practice. In this paper, we propose an Asynchronous Real-Time Federated Learning (ART-FL) approach for anomaly detection in cloud-based microservice systems. In our approach, edge clients perform real-time learning with continuous streaming local data. At the edge clients, we model intra-service behaviors and inter-service dependencies in multi-source distributed data based on a Span Causal Graph (SCG) representation and train a model through a combination of Graph Neural Network (GNN) and Positive and Unlabeled (PU) learning. Our FL approach updates the global model in an asynchronous manner to achieve accurate and efficient anomaly detection, addressing computational overhead across diverse edge clients, including those that experience delays. Our trace-driven evaluations indicate that the proposed method outperforms the state-of-the-art anomaly detection methods by 4% in terms of F_1-score while meeting the given time efficiency and scalability requirements.

INDEX TERMS Anomaly detection, distributed data, federated learning, microservice, trace.

I. INTRODUCTION

INDUSTRIAL microservice systems typically encompass large-scale distributed architectures, housing hundreds to thousands of services within complex cloud infrastructures [1]. These cloud-based systems experience dynamic creation and destruction of service instances, necessitating prompt and efficient analytics capabilities within a bounded time. To address this challenge, machine learning (ML) techniques are increasingly being applied to analyze the vast amounts of real-time data generated by these microservices, offering enhanced monitoring and predictive insights [2]. Achieving such insights requires observability, which involves tracing the flow of requests among services in highly intricate and dynamic microservice environments. To this end, it is imperative to monitor the run-time state of microservices using real-time information such as traces and logs, which record the dynamics of interactions between different services (inter-service interactions) and the activities within individual services (intra-service behaviors), respectively [3]. By doing so, early detection and mitigation of potential anomalies and shortcomings become feasible, thereby averting their progression into consequential failures. The run-time information serves as the foundation for ML-based trace analysis techniques, which aim to comprehend system behaviors [4], detect anomalies [5], [6], and diagnose faults [7] in cloud-based microservice architectures.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

Detection of anomalies in a microservice system is complex due to the following challenges. First, microservice systems are highly dynamic with rapid updates and continuous integration and deployment of new features. These systems often run in a containerized environment, where the state of each container frequently changes from busy to idle, adding to the difficulty of performance diagnosis. Debugging in microservice systems is also challenging due to their intricate and dynamic nature. Developers face the task of analyzing concurrent behaviors across multiple microservices and comprehending the overall system interaction topology. Second, the microservice architecture involves multiple intricately interconnected fine-grained services that are distributed loosely across the system, resulting in complex tracing paths. Additionally, each microservice may have multiple instances to handle requests, thus further increasing the complexity of trace paths. Therefore, tracing and visualizing system executions are necessary for understanding and debugging distributed systems [1].

To tackle the aforementioned challenges, some efforts have been made to apply ML and deep learning (DL) to trace-based anomaly detection in microservice systems [8], [9], [10], [11], [12], [13], [14]. Most of these works rely on a centralized learning (CL) approach, where a central node collects data from distributed devices to train a model. This approach, however, has a number of limitations, including high computation overhead, propagation delay, and privacy concerns due to data transfer to centralized servers [15]. In contrast, decentralized learning approaches, particularly Federated Learning (FL), aim to train models locally in proximity to the data source. This local training allows the raw training data to remain at the source, not requiring external sharing. Consequently, FL reduces the impact of connection, bandwidth, and latency issues often encountered in centralized approaches [2]. Previous FL approaches commonly employ a synchronous protocol such as FedAvg [16], where the server sends the global model to a chosen subset of clients, collects their updates, and merges them through a weighted average technique. However, this strategy incurs substantial costs due to the synchronization step, where the server must wait for all clients to complete their updates before performing the aggregation. The issue of lagging clients, also known as stragglers, is inevitable due to differences in device capabilities and unstable network conditions. As a solution, an asynchronous FL method is proposed, enabling the server to aggregate updates without waiting for slower clients. These asynchronous protocols result in faster convergence, as no participating clients are left idle. In addition, the majority of the existing methods rely on machine learning to extract information from traces, which are instrumental in tracking and characterizing microservice executions by recording key inter-service data, such as request duration. This, however, often leads to the oversight of other valuable data, such as logs and/or metric data. This disparity arises from the difficulties associated with analyzing data from multiple sources, which is more challenging due to the heterogeneity, frequent interactions, and substantial volume of such data. While traces are vital for diagnosing issues, they are not comprehensive enough to identify all types of anomalies. Conversely, various types of data, such as logs, can capture the internal actions and conditions of a service instance when it is invoked. This is referred to as intra-service behavior, and these data types provide useful information that can detect anomalies. For example, given that CPU exhaustion does not have any impact on service latency, it may not be detected in trace data. However, logs can detect this type of anomaly through CPU-related errors and warnings, highlighting the importance of utilizing both logs and traces for effective anomaly detection.

Recently, in [8], we have proposed our so-called Trace-Driven Anomaly Detection (TDAD), which uses a Graph Neural Network (GNN) to enable learning vector representations for traces and Positive and Unlabeled (PU) learning that allows for training a trace-based anomaly detection model with a partially labeled data. Leveraging historical anomalous traces, TDAD trains the anomaly detection model using a small number of labeled anomalous traces along with a relatively large number of unlabeled traces, thus detecting anomalies in a timely manner. While our work [8] addresses the accuracy and real-time aspects of the problem, it does not consider intra-service behaviors of individual services and communication overhead in microservice applications. In this paper, we build on [8] and propose a real-time anomaly detection method to tackle the challenge of managing communication and computation overhead in AI-enabled microservice systems. This is achieved by proposing an Asynchronous Real-Time Federated Learning (ART-FL), which allows the global model to proceed without waiting for gradient information from lagging clients. In our proposed approach, clients perform real-time learning to manage local streaming data. ART-FL enables continuous computation and communication without delays, ensuring better model convergence compared to synchronized FL approaches. This paper emphasizes prediction performance and computational efficiency in FL rather than communication costs or privacy concerns. Furthermore, our approach captures intra-service behaviors by leveraging log messages with the instances involved in a trace, thus allowing for a grasp of the behavior of individual services within the microservices architecture. The key contributions of this paper are summarized as follows:

- We propose a collaborative anomaly detection method, which exploits multi-source distributed data incorporating the hierarchical structure and contextual information of traces and logs to detect anomalous events in real time. We employ the GNN and PU approach to model both intra-service behaviors and inter-service dependencies.
- To ensure a timely model convergence, we propose an asynchronous FL for collaborative training of a global anomaly detection model across microservices. The ART-FL captures inter-client relationships through regularization and a central feature learning module.

We develop a real-time learning procedure that uses a decay coefficient to balance the influence of past and current gradients on clients managing streaming data. Additionally, we propose a dynamic learning approach that adjusts the step size on local clients, mitigating the adverse effects of slower clients on the convergence of the global model.

- After developing a non-parametric unsupervised learning-based threshold setting scheme for anomaly detection, we carry out a comprehensive experimental evaluation on a real-world microservice benchmark to assess the performance of our proposed method in terms of detection effectiveness, detection time, and resource overhead.

The remainder of this paper is structured as follows. Section II presents the background, motivation, and requirements. The related work is presented in Section III. Section IV presents the system model and problem statement. Section IV-A describes our proposed method. Our evaluation results are presented in Section VI. Section VII concludes the paper.

II. BACKGROUND, MOTIVATION AND REQUIREMENTS

In this section, we first introduce the background information and then present a motivating example, which is then used to derive the key requirements.

A. BACKGROUND

1) DISTRIBUTED TRACING

Distributed tracing systems often follow the OpenTracing[1] standard, a language-independent framework for distributed tracing overseen by the Cloud Native Computing Foundation (CNCF). This standard includes an API specification widely supported by various frameworks and libraries [17]. The OpenTracing specification outlines that a trace represents the execution pathway of a request as it traverses through a distributed system. In microservice architectures, a trace describes the journey of a request across various service instances, forming a sequence of service calls. Initially, individual service instances record these service calls within a trace. These records are subsequently aggregated to assemble a complete trace. Furthermore, system events trigger the generation of logs. A log message, or log, is essentially a record produced by logging statements, comprising both constant strings set by developers and variable values determined by the system's state at the event's occurrence.

In distributed systems, synchronous invocation halts a component until it receives a response, whereas asynchronous invocation lets a component proceed without awaiting a reply, enhancing system availability [18]. Parallel invocation concurrently executes multiple requests, improving throughput and reducing response times, crucial for high-performance systems. Each trace's span represents a service invocation, with synchronous methods potentially exacerbating downtime effects. Conversely, asynchronous methods use message-based communication for better resilience, and parallel methods leverage multi-threading to expedite service responses [18].

Causal relationships in distributed systems can be effectively modeled using SCGs, which represent interactions between entities over time. SCGs use nodes to denote events or states and directed edges to capture causal dependencies. This is particularly valuable in microservice architectures for root-cause analysis of anomalies. For instance, if Service C in a call chain fails, the SCG can trace the causal path back to Service A. Fig. 1 illustrates an example of an SCG from the TrainTicket application, where nodes represent microservices and edges denote their interactions.

2) PU LEARNING

PU learning deals with binary classification when only positive (P) and unlabeled (U) data are available. It is commonly applied in scenarios where negative samples are unavailable, focusing on identifying patterns in the unlabeled data. Trace anomaly detection involves three types of traces: unlabeled normal trace, labeled anomalous trace, and unlabeled anomalous trace. Unlabeled normal traces are generated during normal system operations. Labeled anomalous traces represent anomalies that occur during request execution. Unlabeled anomalous traces are anomalies that remain unlabeled.

B. MOTIVATION AND REQUIREMENTS

To motivate our work with an example, we consider TrainTicket [1], which is a microservice-based application. As shown in Fig.1, a trace is an array of spans arranged in a tree-like hierarchy, where each span represents a service invocation. Traces and spans are uniquely identified by their IDs. A span records information about the caller and callee, including an operation name that specifies the action performed, such as "GET /api/v1/auth/login" in a REST call. Except for the root, each span is linked to a parent span that initiates its invocation. For example, Span 001 triggers two parallel synchronous calls to Spans 002 and 003, as illustrated in Fig. 1. Fig. 2 depicts the timeline of the spans (shown in Fig. 1). Spans 002 and 003, both originating from Span 001, are executed in parallel, leading to a temporal overlap in their execution timelines (see Fig. 2). Meanwhile, Span 005, an asynchronous invocation initiated by Span 003, is completed after the completion of Span 003.

Microservice architectures can face various anomalies like unexpected service failures, performance issues, configuration errors, coding mistakes, and coordination problems, disrupting system stability. Current trace anomaly detection methods [12], [13], [14] view traces as service interaction sequences, and log anomaly detection methods [19], [20], [21] treat them as log event series. However, applying these methods effectively in microservices is challenging due to several reasons. First, detecting anomalies requires aggregating logs from multiple services, but existing trace methods overlook log data, limiting their detection capabilities to

[1] See https://opentracing.io/ for further information.

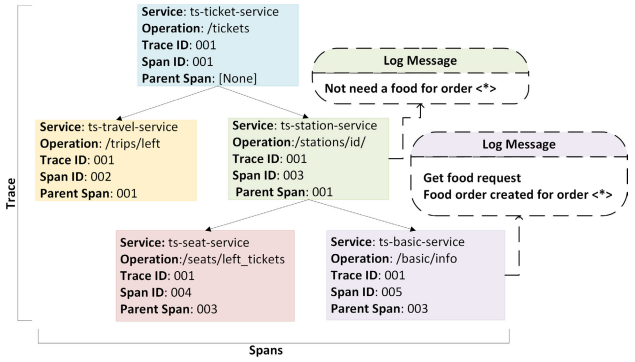


FIGURE 1. Illustration of trace, spans, and logs within the TrianTicket microservice application.

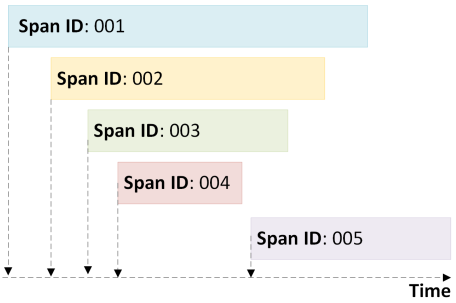


FIGURE 2. Timeline of Spans in Fig. 1.

anomalies within trace structures alone. Fig. 1 highlights a log-level anomaly where Span 003's logs indicate no food request for a train ticket, yet Span 005's logs show a food order for that ticket. Detecting such anomalies necessitates merging logs from both Spans 003 and 005. Second, traces have complex structures, including hierarchical, parallel, and asynchronous calls, which are not fully captured by sequential representations. This simplification misses the causal relationships and the temporal dynamics within and across spans. For instance, in the sequence-based representation, two adjacent log events within Span 001 may appear distant due to the insertion of log events from Span 001's descendant spans. Furthermore, traces often involve parallel or asynchronous service calls. Figs. 1 and 2 show a trace with an asynchronous (Span 005) and two parallel calls (Spans 002 and 003). This setup leads to the interleaving of log events from Spans 002, 003, and their children, as well as between Span 005 and Span 003, in varied sequences. Merging these events into a single chronological sequence obscures the unique aspects of parallel and asynchronous calls, emphasizing the need to maintain trace structure in log integration.

Although traces are commonly used in anomaly detection [12], [13], [14], they are found to be inadequate for identifying all potential anomalies. They primarily provide a broad view of the system and record interactions across microservices. This high-level data allows for basic, coarse-grained queries, but lacks detailed intra-service data. For example, granular details like memory usage, critical for assessing the internal state of a service, are absent in trace data. This limitation underlines the fact that network-related issues, such as latency, cannot wholly encapsulate anomalies associated with resource constraints. In such scenarios, the collection of multi-source data provides valuable information, enabling real-time detection of anomalous behaviors. However, as illustrated in the earlier example, two important challenges emerge. First, detecting anomalies in microservice systems poses significant difficulties due to their dynamic nature, frequent updates within containerized environments, and complex interconnections among fine-grained services. Second, in a microservices environment, the data is expected to be widely distributed, where communication often becomes a significant bottleneck. Clearly, the cloud microservice environment requires the anomaly detection system to be (i) real-time, (ii) accurate, (iii) scalable, and (iv) capable of maintaining low communication and computation overhead while effectively handling multi-source distributed data.

III. RELATED WORK

In this section, we review existing approaches for anomaly detection in centralized learning and federated learning across various applications and domains.

A. CENTRALIZED LEARNING ANOMALY DETECTION

Gan et al. [9] developed Seer, a performance anomaly detection method for services that integrates convolutional and LSTM layers. In [10], dual neural networks for service anomaly detection were proposed. Their method involves a variational autoencoder, trained on normal activity to spot anomalies through reconstruction errors, and a convolutional neural network trained on failure-injected data for specific anomaly detection. This approach effectively reduces false positives and identifies anomaly types in anomalous services. Bogatinovski et al. [11] developed a self-supervised encoder-decoder network for anomaly detection by learning event dependencies. In [3], SCWarn employed a multi-model LSTM approach to analyze data from diverse multi-source inputs.

The authors of [12] proposed TraceAnomaly, which utilizes a deep Bayesian neural network with posterior flow for anomaly detection. Nedelkoski et al. [13] proposed a multi-modal LSTM neural network for anomaly detection in multi-service applications trained on normal execution traces. Microscope [14] is a performance anomaly detection model for applications, tracking front-end KPIs of microservices against predefined Service Level Objectives (SLOs). Jin et al. [5] proposed their so-called RPCA, which is an offline anomaly detection method for microservices using distributed traces. Centralized methods for anomaly detection, while effective, face practical challenges in distributed edge device scenarios. First, these models often lose accuracy over time due to their inability to adapt to evolving data. Second, privacy concerns hinder data exchange among edge devices, leading to the creation of the so-called ‘‘data

TABLE 1. Comparison of existing anomaly detection methods with the proposed method.

Related Work	Domain	Data Type	Learning Manner	Requirements			
				Real-Time	Accurate	Scalable	Low Overhead
DIoT [22]	Intent of Things (IoT)	Metric	FL	✗	✓	✓	✓
FSLSTM [23]	Smart Building	Log	FL	✗	✓	✗	✓
FL-GRU [24]	Industrial Control System	Metric	FL	✗	✓	✓	✓
FATRAF [25]	Industrial Control System	Metric	FL	✗	✗	✓	✓
FLOGCNN [19]	Software Systems	Log	FL	✗	✓	✗	✓
DeepFed [26]	Cyber Physical System	Metric	FL	✗	✓	✗	✓
FedAnomaly [27]	Cloud Manufacturing	Metric	FL	✓	✗	✗	✓
TCN-ACNN [28]	IoT	Log	FL	✓	✗	✗	✓
TraceAnomaly [12]	Microservice Application	Trace	CL	✗	✗	✓	✗
MultimodalTrace [13]	Microservice Application	Trace	CL	✗	✗	✓	✗
LogAnomaly [20]	Software Systems	Log	FL	✗	✓	✗	✓
RobustLog [21]	Software Systems	Log	CL	✗	✓	✓	✗
Microscope [14]	Microservice Application	Trace	CL	✓	✓	✓	✗
FL-SmartGrid-AD [29]	Smart Grid	Metric	FL	✓	✓	✗	✓
TDAD [8]	Microservice Application	Trace	CL	✓	✓	✗	✗
SCWarn [3]	Microservice Application	Metric and Log	FL	✓	✓	✗	✓
Seer [9]	Microservice Application	Metric and Trace	CL	✗	✓	✓	✗
Proposed Method	Microservice Application	Trace and Log	FL	✓	✓	✓	✓

islands," which impairs anomaly detection efficiency. Third, centralized machine learning methods are not able to satisfy both the accuracy and real-time requirements of anomaly detection.

B. FEDERATED LEARNING ANOMALY DETECTION

Nguyen et al. [22] used FL for network intrusion detection in IoT devices. In [23], the authors used FL and LSTM to monitor activities and detect abnormal energy usage in smart homes. Truong et al. [25] developed a fast, resource-efficient FL method for anomaly detection in industrial control systems. Mothukuri et al. [24] employed FL with federated Gated Recurrent Unit (GRU) models for detecting and classifying attacks in IoT networks. In [30], the authors introduced a federated learning-based system for anomaly detection in IoT streaming data, incorporating a compression algorithm to improve FL gradient communication efficiency and reduce network resource use. In [31], an FL-based anomaly detection method using autoencoder architecture was proposed for IIoT. In [32], an FL method for IoT anomaly detection was proposed, which integrates differential privacy for enhanced user privacy. In [19], an FL-based approach named FLOG was proposed to detect anomalies in distributed log data. Deepfed, a FL-based framework for intrusion detection in cyber-physical systems (CPSs), was introduced in [26]. Convolutional neural networks (CNNs) and GRUs are combined in this architecture to detect threats. Li et al. [28] applied deep learning for extracting features from log sequences and employed an FL framework for training in a distributed IoT environment.

FedAnomaly, an FL-based transformer framework, was proposed in [27] for anomaly detection in cloud manufacturing. This method enables distributed edge devices to jointly train a global model without sharing data, thus preserving privacy and enhancing anomaly detection in cloud manufacturing settings. Kong et al. [33] introduced a federated graph anomaly detection framework that leverages contrastive learning and anomaly information updates to enhance detection performance on distributed graph data. Jithish et al. [29] developed an FL approach for anomaly detection in smart grids, focusing on enhancing data security and privacy.

Table 1 compares the existing works with our paper based on our requirements. Centralized learning in microservices uses supervised or unsupervised algorithms [9], [10], [11], [12], [14]. Unsupervised learning, assuming most data is normal, fails to effectively incorporate historical anomalies, while supervised learning requires extensive, resource-heavy labeling, potentially missing diverse anomalies. Additionally, trace comparison methods are useful for anomaly detection but may not meet real-time detection needs due to their slowness [7], [34]. While existing research uses deep learning for metric or log analysis in microservice anomaly detection, it often neglects distributed trace data's potential [12], [13], [14]. These studies simplify traces to sequences of service calls, ignoring their complex structure, including hierarchical, parallel, and asynchronous relationships, and the detailed log messages critical for anomaly detection. Furthermore, current log anomaly detection methods [20], [21] mainly learn from normal operations, identifying deviations as anomalies without capturing the subtle anomaly patterns effectively. The existing FL approaches [27], [29] assume uniform device behavior under synchronous protocols, which does not accurately reflect the real-world heterogeneity of devices. These approaches struggle with variations in data volume and distribution, system latency, hardware configurations (such as memory and processor speed), and the vailability issue of edge devices. This leads to inconsistent computation times and contributions across clients in the FL model. The research gap explained above highlights the necessity for asynchronous FL approaches to accommodate device heterogeneity, to develop efficient real-time detection methods for operational microservice systems, and to propose scalable solutions for handling multi-source distributed data.

IV. SYSTEM MODEL AND PROBLEM STATEMENT

In this section, we first describe our system model and then present the problem statement.

A. SYSTEM MODEL

In large-scale industrial microservice systems, thousands of microservices across distributed machines generate billions of runtime data points daily, including traces and logs. Our system model involves two main entities, (i) participants, who act as clients, and (ii) a cloud server. Let $\mathcal{K} = \{1, \ldots, K\}$ represent the set of K clients, where each client has a distinct dataset $\mathcal{D}_{k\in\mathcal{K}}$ and represents $n_k = |\mathcal{D}_k|$ to be the number of samples on client k. We denote $N = \sum_{k=1}^{K} |\mathcal{D}_k|$ as the total number of samples in K clients. Each client k independently trains a local model $\mathbf{w}_k$ using its dataset $\mathcal{D}_k$. Notably, only the local model parameters are transmitted to the cloud server. This process deviates from traditional centralized training, where datasets are aggregated, $\mathbf{D} = \cup_{k\in\mathcal{K}}\mathcal{D}_k$, before model training. The local models, represented by $\mathbf{w}_k$, collectively contribute to the generation of a global model $\mathbf{w}_G$ through aggregation at the server. In this system, clients act as collaborative participants, working together to train a machine learning model, which is coordinated by an aggre-

TABLE 2. Summary of main notations.

Symbol	Definition
$\mathbf{w}_G$	Global model parameters
$\mathbf{w}_k$	Local model parameters for client k
π_p	Class prior probability of anomalous traces
T_z	Candidate threshold for anomaly detection
α_{ij}	Attention coefficient from node j to i in the graph
σ	Standard deviation
N	Total number of samples
$\ell_i(x_i, y_i; \mathbf{w}_k)$	Loss function for the data point
η_t	Learning rate at iteration t
λ	Regularization parameter
Γ	Decay coefficient
T_e^0	Initial threshold for anomaly detection
$F(\mathbf{w}_G)$	Global objective function
ξ_k	Gradients computed during local updates
r_t	Multiplier for dynamic learning rate
y_i	Output of the MLP for a given instance i.
q_i	Anomaly probability for instance i.
T_e	Anomaly detection threshold.
T_z'	Updated candidate threshold in refinement.
d	Step size for candidate threshold generation.
C_x	Cluster assigned by K-means.
$\mathcal{Q}$	Set of all anomaly probabilities.
$\mathcal{F}$	K-means clustering function.
$\mathcal{C}$	Set of all clusters.
$\mathcal{Q}_A$	Subset of anomaly probabilities above the threshold.
$\mathcal{Q}_N$	Subset of anomaly probabilities below the threshold.

gate server. The symbols used in the following sections are summarized in Table 2 for reference.

B. PROBLEM STATEMENT

In a large-scale system comprising M microservices, each microservice aggregates logs and traces separately. As previously discussed, there are K clients, each running an application with M microservices. This means that there are multiple instances of the application, each managed by a different client. In an observation window of length T (where data collected in this window forms a sample), we define multi-source data as $\mathbf{X} = \{(\mathbf{X}_m^{\mathcal{L}}, \mathbf{X}_m^{\mathcal{T}})\}_{m=1}^{M}$, where $\mathcal{L}$ represents the collection of log events from all microservices. Therefore, $\mathbf{X}_m^{\mathcal{L}}$ is specific to the microservice m and denotes the log events associated with it. $\mathcal{T}$ indicates the collection of trace records from all microservices. Therefore, $\mathbf{X}_m^{\mathcal{T}}$ represents the trace records for the microservice m during the observation window T.

Let us define $\mathbf{w}_k$ as the local model for client k. We assume that for any $k \neq k'$, we have $\mathcal{D}_k \cap \mathcal{D}_{k'} = \emptyset$. The local empirical loss for client k is then given by:

$$f_k(\mathbf{w}_k) \stackrel{\text{def}}{=} \frac{1}{n_k} \sum_{i \in \mathcal{D}_k} \ell_i(x_i, y_i; \mathbf{w}_k), \tag{1}$$

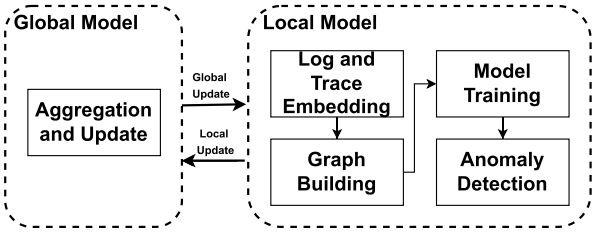


FIGURE 3. High-level overview of MS-FLAD.

where $\mathbf{w}_k$ is the local model parameter and $\ell_i(x_i, y_i; \mathbf{w}_k)$ represents the loss function for the data point $\{x_i, y_i\}$. The global objective function is obtained as follows:

$$F(\mathbf{w}_G) = \sum_{k=1}^{K} \frac{n_k}{N} f_k(\mathbf{w}_G), \tag{2}$$

where $\mathbf{w_G}$ is the aggregated global model. The overall goal is to find the model $\mathbf{w}_G^*$ that minimizes $F(\mathbf{w}_G)$:

$$\mathbf{w}_G^* = \arg\min F(\mathbf{w}_G). \tag{3}$$

With these considerations in mind, the goal at the local client is to solve the problem of determining whether $\mathbf{X}_{[1:M]}$ is an anomaly or not in real time.

V. MICROSERVICE FEDERATED LEARNING ANOMALY DETECTION (MS-FLAD)

Fig. 3 provides a high-level overview of our proposed method, which is called Microservice Federated Learning Anomaly Detection (MS-FLAD), highlighting the interaction between the Global Model and the Local Model. The Global Model performs federated aggregation and updates asynchronously. The Local Model consists of four main components: Log and Trace Embedding, Graph Building, Model Training, and Anomaly Detection. Fig. 4 illustrates the detailed workflow of the Local Model in MS-FLAD. The process begins with Log and Trace Embedding, which extracts semantic, temporal, and status code features. These embeddings are used to construct SCGs in the Graph Building phase. The Model Training component utilizes GAT and PU learning to generate trace representations. Finally, the Anomaly Detection component identifies anomalies by applying a threshold to the computed anomaly probabilities. Each module is described in detail below.

A. LOG AND TRACE EMBEDDING

In this section, we describe the log embedding process, which parses raw log messages. Next, semantic vectorization turns these messages into fixed-dimension vectors. Then, we focus on tracing embedding, which is essential for understanding system causal relationships and extracting vectors for each span.

1) LOG EMBEDDING

Given that raw log messages have an unstructured nature, automated log analysis encounters challenges. Unstructured

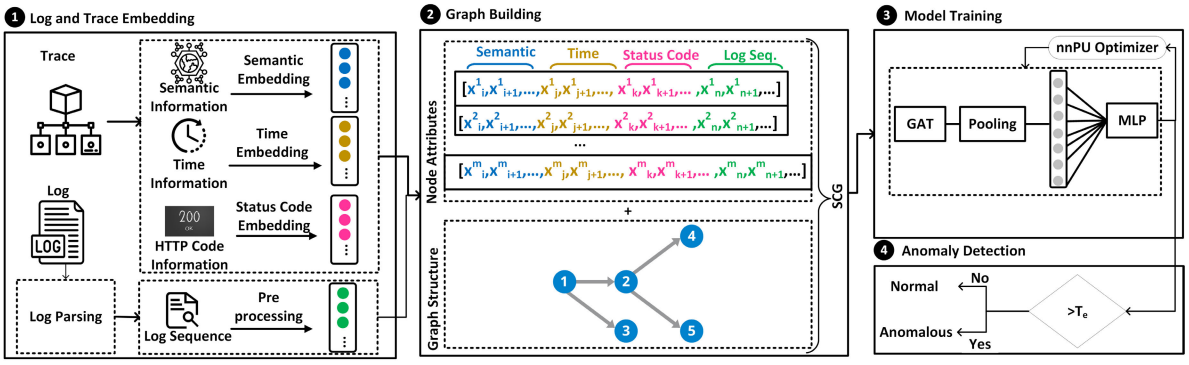


FIGURE 4. Overview of the local model method for the edge clients.

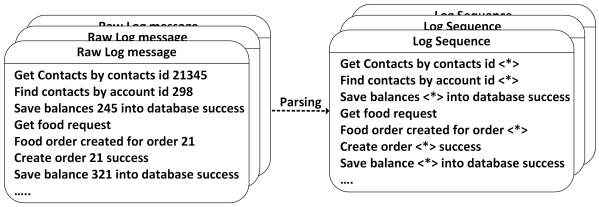


FIGURE 5. Employing the drain method for log parsing on raw log messages from the trainticket application.

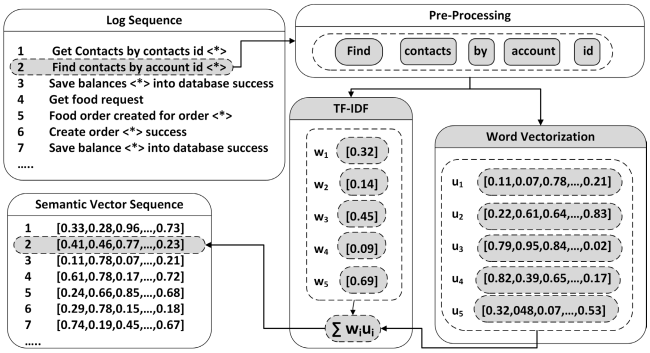


FIGURE 6. Workflow of semantic vectorization.

log messages introduce difficulties in parsing and extracting meaningful information, attributed to the lack of standardization. The inherent variability, limited searchability, and reduced readability further compound these challenges, hindering the efficiency of log analysis processes. To address this, a log parsing strategy is employed to abstract the parameters within each log message. This transforms the data into a structured format, facilitating subsequent analysis. We adopt the so-called Drain [35], a technique known for its high parsing accuracy and efficiency in processing unstructured log data in real-time streams. To facilitate an integrated analysis with trace data, we first identify and capture the span ID and trace ID linked to each log message before initiating the parsing process. Following the parsing step, both the trace ID and span ID are appended to every extracted log event.

Fig. 5 presents a log data example from the TrainTicket application, simplifying some fields for clarity [1]. Log messages, shown on the console, include a consistent log event and a variable log parameter, the latter capturing attributes like contact ID and balance value. Log parsing, as executed by Drain, extracts log events from messages, facilitating their conversion into structured sequences, as depicted in Fig. 5 a log sequence consists of log events that track a specific task's execution flow. Each event in the sequence shares the same task ID, allowing for their chronological association.

2) SEMANTIC VECTORIZATION

Semantic vectorization extracts semantic information from log events, transforming it into a fixed-dimension vector, known as a semantic vector. This transformation takes place independently of the log event's previous occurrences. The semantic vectorization workflow is depicted in Fig. 6 and encompasses three key steps: log events pre-processing, word vectorization, and TF-IDF-based aggregation, which are explained in more detail next.

a: LOG EVENTS PRE-PROCESSING

To determine the semantics of log events, we treat each log event E as a natural language sentence, expressed as $S = [t_1, t_2, \ldots t_N]$, where token i is represented by t_i, $\forall i \in [1, N]$ and N indicates the total length of the log event sentence. Although most tokens are valid English words, log events may contain non-character tokens and various variable names. The pre-processing for each log event sentence involves several steps. First, we eliminate all non-character tokens from sentences S, including delimiters, operators, punctuation, and numerical digits. Then, we discard common stop words such as "a" and "the". Finally, we split composite variable names in log events into their basic components, e.g., "TypeDeclaration" becomes "type" and "declaration", and "isCommitable" splits into "is" and "Commitable". This method deconstructs compound tokens into their elemental parts.

b: WORD VECTORIZATION

After pre-processing, each log-event sentence S is converted into a semantic vector V. This transformation leverages pre-trained word vectors from FastText to capture the semantic meaning of log events, including semantic similarity among words. FastText assigns each word a D-dimensional

vector, with $D = 300$ for its word vectors. Consequently, a log-event sentence S is transformed into a list of word vectors $\mathcal{U} = [\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots, \boldsymbol{u}_N]$, where each word is replaced by its corresponding vector. The word vector for t_i is represented as $\boldsymbol{u}_i \in \mathbb{R}^d$, $i \in [1, N]$, and N represents the total number of tokens in the log-event sentence.

c: TF-IDF AGGREGATION

To ensure a fixed-dimensional representation for log events, which may vary in word count N, we aggregate the N word vectors in $\mathcal{U}$ into a D-dimensional vector. This method guarantees the uniform D for all semantic vectors, even with different N values. In log events, not all words carry the same level of significance. Commonly occurring words, such as ''api'' and ''get'', are less crucial compared to less frequent words like ''food'' and ''submit'' in the context of log embedding. Therefore, we apply weighted aggregation using the Term Frequency-Inverse Document Frequency (TF-IDF) [36]. This method ensures uniform dimensionality across all semantic vectors while emphasizing the significance of less frequent words. The TF-IDF weight for each word is computed as: $\text{TF-IDF} = \text{TF} \times \text{IDF}$, where $\text{TF(word)} = \frac{\text{Frequency of word in } S}{N}$ and $\text{IDF(word)} = \log\left(\frac{L'}{L'_{\text{word}}}\right)$. Here, L' is the total number of log events, and L'_{word} is the number of events containing the word. The semantic vector $U \in \mathbb{R}^d$ for a log event is then computed by aggregating word vectors $\mathcal{U}$ using TF-IDF weights: $U = \sum_{i=1}^{N} w_i \cdot \boldsymbol{u}_i$, where w_i is the TF-IDF weight of the i-th word, $\boldsymbol{u}_i \in \mathbb{R}^d$ is its vector representation, and the dot operation $(\cdot)$ denotes scalar multiplication. This approach captures semantic similarities between log events while distinguishing them effectively.

3) TRACE EMBEDDING

A trace consists of multiple spans that illustrate the causal relationships between different service calls. Each span corresponds to a specific service invocation and captures information such as the service and operation names, start time, duration, and status code. Span embedding refers to the process of capturing and encoding these invocation details to generate a trace graph representation. This involves encoding service and operation names, start and duration times, and status codes individually. These elements are then combined into a vector format for each span, providing a foundation for further analysis and interpretation of the trace data. Span embedding is carried out in three primary stages: (i) Semantic Embedding, (ii) Time Embedding, and (iii) Status Code Embedding, which are described in more detail next.

a: SEMANTIC EMBEDDING

The semantic embedding component is responsible for producing a vector representation for each span in a trace by combining the service name and operation name. To start, we split these names into individual words using common delimiters found in microservices, such as ''/'', ''-'', and '':''. All words are then converted to lowercase, and non-alphabetical symbols, including punctuation marks and numbers, are removed. For example, the name ''ts-travel-service/POST:/api/v1/travelservice/travelPlan/cheapest'' is segmented into ''ts'', ''travel'', ''service'', ''post'', ''api'', ''v1'', ''travelservice'', ''travelplan'', and ''cheapest''. To manage the dynamic and variable nature of service and operation names in microservices, we apply the WordPiece tokenization algorithm [37], which divides words into smaller tokens based on character sequences. This technique helps handle out-of-vocabulary (OOV) words. For instance, ''traveldate'' and ''trainnumber'' are tokenized into ''travel'', ''date'', ''train'', and ''number''. Using the token sequence obtained from this process, we then utilize a pre-trained BERT model [38], a transformer-based language representation model, to generate vector embeddings. Specifically, we use the BERT-Base model [38], which includes 12 transformer encoder layers and has a hidden layer size of 768 dimensions. This model generates a 768-dimensional vector for each span, capturing the semantic details of both the service and operation names. Leveraging the BERT-Base model's 12-layer transformer architecture and its 768-dimensional hidden layer has proven effective for comprehending complex language patterns and enhancing performance in tasks requiring detailed semantic understanding.

b: TIME EMBEDDING

Each span in the system records crucial details about a service invocation, such as the start time and duration of the interaction between the caller and callee. To improve the detection of time-related anomalies, we extract four key time-based attributes from these timestamps: (1) duration time, representing the total time of the span; (2) waiting time, which is the period the callee waits for a response from other services; (3) local execution time, referring to the time taken by the callee to complete the current invocation, excluding the waiting time; and (4) relative start time, which measures the time difference between the start of the span and the start of the relevant parent span.

With the BERT-Base model, each span is represented by a 768-dimensional vector that captures the embedding of both the service and operation names. However, if time features are limited to just one dimension within this 768-dimensional vector, there is a risk that the time-related characteristics could be underrepresented in the span's overall embedding. Moreover, the significant variation in time durations across different traces (ranging from just a few milliseconds to several thousand) complicates the process of weight convergence and reduces the model's training efficiency. To address these issues, we project a single-dimensional time feature $\boldsymbol{t}$ into a 768-dimensional vector space denoted by $\boldsymbol{E}_{\text{time}}$. Subsequently, we use the softmax function to create a soft one-hot encoding, where each element lies between 0 and 1, and the sum of all elements equals 1. The soft one-hot encoding vector $\boldsymbol{s}$ is obtained as $\mathbf{s} = \tau(\boldsymbol{t}\boldsymbol{W} + \boldsymbol{b})$, where $\tau(\cdot)$ is

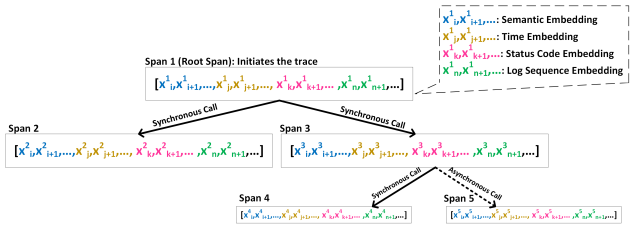


FIGURE 7. **An Example of an SCG in TrainTicket application.**

the softmax function, $\boldsymbol{W} \in \mathbb{R}^p$ is the weight matrix, and $\boldsymbol{b} \in \mathbb{R}^p$ is the bias. Next, we project the vector $\mathbf{s}$ into a vector space specifically designed for time embeddings. The soft one-hot encoding $\mathbf{s}$ is multiplied by the time embedding vector $\boldsymbol{E}_s \in \mathbb{R}^{p\times d}$, which results in p-dimensional vector $\boldsymbol{E}_{\text{time}}$ as $\boldsymbol{E}_{\text{time}} = \mathbf{s} \odot \boldsymbol{E}_s$, where $\odot$ denotes the element-wise multiplication of two vectors of the same length. Finally, to create a comprehensive representation of the time-related attributes within a span, we concatenate the four time embedding vectors, i.e., duration time, waiting time, local execution time, and relative start time (which are defined above). This combined representation effectively captures and encodes the temporal information associated with the span.

c: STATUS CODE EMBEDDING

The HTTP/1.1 standard [39] defines a total of 63 status codes, organized into five main categories. We apply one-hot encoding to represent these status codes, where each status code is mapped to a 63-dimensional vector. In this representation, each dimension corresponds to a specific status code. For example, a status code of 200 is encoded as a vector with a value of 1 in the 5th dimension and 0 in all other dimensions, while a status code of 404 is encoded with a value of 1 in the 28th dimension and 0 in all other dimensions. This method of encoding status codes enables an efficient analysis of HTTP traffic.

B. GRAPH BUILDING

Traces have a hierarchical structure consisting of service invocations, known as spans. In our proposed MS-FLAD method, this hierarchical structure is efficiently modeled using an SCG, a directed acyclic graph. In this graph, each node corresponds to a span within the trace, while the edges represent the parent-child relationships between spans. A directed edge from Span 1 to Span 2 signifies that Span 1 is the parent of Span 2. The vector representation of each span is assigned as an attribute to the corresponding graph node to capture its specific characteristics. Fig. 7 illustrates an example of an SCG, which corresponds to the trace shown in Fig. 4. In this figure, each rectangle represents a span, and each edge signifies a causal relationship between two spans. For instance, Span 1 represents the root span of the trace, and Spans 2 and 3 are child spans of Span 1. Each span's vector representation comprises four components, i.e., semantic, time, status code, and log sequence.

C. MODEL TRAINING

In our proposed approach, trace-based anomaly detection is formulated as a PU learning problem, leveraging historical knowledge of anomalous traces while minimizing the number of labeled traces used for training. PU learning involves training a binary classifier with a small set of positive samples (i.e., anomalous traces) and a larger set of unlabeled samples. Traces are represented as SCGs, where span and log embeddings are used as node attributes. To derive meaningful trace representations, we utilize a Graph Neural Network (GNN) known as the Graph Attention Network (GAT), which incorporates the multi-head self-attention mechanism [40]. We selected GAT because it is well-suited to the graph structure of trace data, enabling it to learn vector representations of traces while capturing complex relationships and dependencies within the graph. These graph neural networks produce vector representations of the traces, as illustrated in Fig. 4. For training the binary classifier in trace-based anomaly detection, we adopt the non-negative risk estimator (nnPU) algorithm [41], which is known for its robustness against overfitting.

Let $g = V, A, X$ represent an SCG, where V is the collection of nodes, A is the adjacency matrix that defines the edges, and X is the set of attributes for each node, with x_i denoting the vector representation of each node. In an SCG, The GAT layer computes attention scores to prioritize neighboring nodes as follows: $e_{ij} = \psi\left(\mathbf{a}^T \cdot \left(\mathbf{W}\mathbf{h}_i \| \mathbf{W}\mathbf{h}_j\right)\right)$, where $\psi(\cdot)$ is the LeakyRelu activation function, $\|$ represents concatenation, $\mathbf{h}_i$ and $\mathbf{h}_j$ denote the vector representations of node i and node j, respectively. The weight matrix $\mathbf{W} \in \mathbb{R}^{F'\times F}$ corresponds to a shared linear transformation, where F is the dimensionality of the input node features and F' is the dimensionality of the transformed features. $\mathbf{a}$ is a learnable attention vector. To calculate the attention coefficients α_{ij} from node j to node i, the importance across all neighboring nodes of i is normalized using a softmax function, expressed as: $\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{c\in\mathcal{N}_i}\exp(e_{ic})}$, where $\mathcal{N}_i$ represents the neighborhood of node i. The GAT utilizes multi-head attention to enhance the stability of the attention mechanism's learning process. The attention coefficients are used to compute the output node representation $\mathbf{h}'_i$ as $\mathbf{h}'_i = \|_{c=1}^{C}\sigma\left(\sum_{j\in\mathcal{N}_i}\alpha_{ij}^c\mathbf{W}^c\mathbf{h}_j\right)$, where C denotes the number of attention heads involved. Each attention head, represented by α_{ij}^c, has its own attention score. The weight matrix $\mathbf{W}^c$ corresponds to the linear transformation for attention head c. $\sigma(\cdot)$ refers to the activation function.

After executing the computations through m GAT layers, MS-FLAD generates vector representations for each node. The overall graph representation, v_g, is derived by averaging the vectors across all nodes expressed as: $v_g = \frac{1}{N_g}\sum_{n=1}^{N_g}\mathbf{h}_n^m$, where N_g denotes the number of nodes in graph g. The vector representation $\mathbf{h}_n^m$ represents the output of node n from the embeddings of GAT layer m.

During the training phase, our proposed MS-FLAD utilizes non-negative risk estimation from the nnPU algorithm [41],

a large-scale PU learning technique, to iteratively optimize the GAT parameters. In each epoch, the training dataset is split into N mini-batches, and MS-FLAD updates the GAT parameters based on the risk estimation for each mini-batch. Consider a two-layer perceptron (MLP) function f with a single output dimension, and let L represent the sigmoid loss function. Each mini-batch contains n_p labeled anomalous traces, where p denotes positive (anomalous) data, and u refers to the unlabeled data. The estimated risk $\widehat{R}_{\mathrm{p}}^{+}$ associated with labeled anomalous traces is obtained as follows:

$$\widehat{R}_{\mathrm{p}}^{+} = \frac{1}{n_p} \sum_{i=1}^{n_p} L\left(f\left(v_i^p\right), +1\right). \tag{4}$$

Similarly, the estimated risk $\widehat{R}_{\mathrm{p}}^{-}$ associated with misclassifying labeled positive (anomalous) traces as negative is obtained as follows:

$$\widehat{R}_{\mathrm{p}}^{-} = \frac{1}{n_p} \sum_{i=1}^{n_p} L\left(f\left(v_i^p\right), -1\right). \tag{5}$$

We calculate the estimated risk $\widehat{R}_{\mathrm{u}}^{-}$ for unlabeled traces by treating them as normal as follows:

$$\widehat{R}_{\mathrm{u}}^{-} = \frac{1}{n_u} \sum_{i=1}^{n_u} L\left(f\left(v_i^u\right), -1\right), \tag{6}$$

where n_u is the number of unlabeled traces. Finally, the empirical risk estimation $\widehat{R}_{\mathrm{pu}}$ is given by:

$$\widehat{R}_{\mathrm{pu}} = \pi_{\mathrm{p}} \widehat{R}_{\mathrm{p}}^{+} + \widehat{R}_{\mathrm{u}}^{-} - \pi_{\mathrm{p}} \widehat{R}_{\mathrm{p}}^{-}, \tag{7}$$

where the hyperparameter π_{p} denotes the class prior probability of anomalous traces.

Let β be the hyperparameter which ensures the risk is non-negative. If $\widehat{R}_{\mathrm{u}}^{-} - \pi_{\mathrm{p}} \widehat{R}_{\mathrm{p}}^{-} \geq -\beta$, MS-FLAD uses Adam [42] optimization to minimize $\widehat{R}_{\mathrm{pu}}$ and optimize GAT parameters; otherwise, Adam is used to minimize $\pi_{\mathrm{p}} \widehat{R}_{\mathrm{p}}^{-} - \widehat{R}_{\mathrm{u}}^{-}$ and optimize the GAT parameters.

D. ANOMALY DETECTION

The anomaly detection component is responsible for generating the final outcome. To detect anomalies, we apply the sigmoid function to the output of the MLP. Let y_i be the output of the MLP for $f\ (v_i)$. The anomaly probability q_i is given by:

$$\mathrm{q}_i = \sigma\ (y_i) = \frac{1}{1 + e^{-y_i}}, \tag{8}$$

To detect anomalies, a threshold T_e is considered; if $q_i > T_e$, an anomaly is detected.

1) INITIAL THRESHOLD SETTING

We build a set of anomaly probabilities $\mathcal{Q} = \{q_1, q_2, \ldots, q_{|\mathcal{Q}|}\}$, where $q_i \in \mathcal{Q}$, $\forall i \in \{1, \ldots, |\mathcal{Q}|\}$ and we define K-means clustering function $\mathcal{F}(q_i)$ that maps each q_i to a cluster $C_x \in \mathcal{C}$, as follows:

$$\mathcal{F}(q_i) = C_x,\ \forall i \in \{1, \ldots, |\mathcal{Q}|\},\ \forall x \in \{1, \ldots, |\mathcal{C}|\}. \tag{9}$$

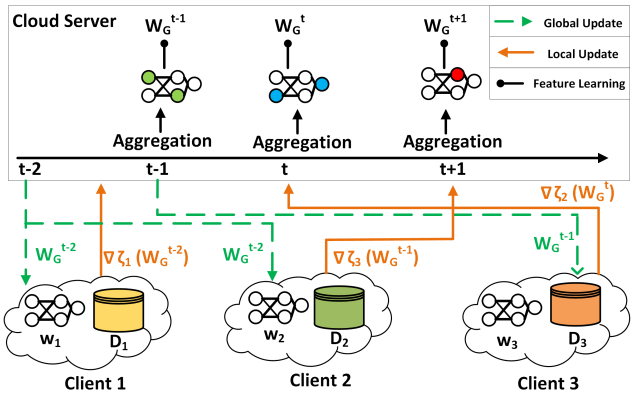


FIGURE 8. ART-FL overview.

Depending on the value of q_i, the system can fall in one of the following states: (i) normal, (ii) transitive, or (iii) anomalous. We set the initial cluster centers to 0, the average of all q_i in $\mathcal{Q}$, and the maximum q_i value in $\mathcal{Q}$ for normal, transitive, and anomalous states, respectively. Let $\mathcal{Q}_a$ be the set obtained for the anomalous cluster. The initial threshold T_e^0 is then obtained as follows:

$$T_e^0 = \min(\mathcal{Q}_a). \tag{10}$$

2) THRESHOLD CALCULATION

To refine the anomaly detection, we generate a series of candidate thresholds T_z as:

$$T_z = T_e^0 + z \times d,\ \forall z \in \{0, \ldots, z_{\max}\} \tag{11}$$

where z is the smallest integer such that $T_z > max(Q)$, and d is the step size. Large values of d may increase false positives, while smaller values of d may increase the execution time of the algorithm. For each candidate threshold T_z, the set $\mathcal{Q}$ is divided into two subsets, $\mathcal{Q}_N = \{q_i \in \mathcal{Q} \mid q_i \leq T_z\}$ and $\mathcal{Q}_A = \{q_i \in \mathcal{Q} \mid q_i > T_z\}$. The updated T_z' is then calculated for each T_z as follows:

$$T_z' = \frac{\sigma(\mathcal{Q})}{\sigma(\mathcal{Q}_N)} \times \frac{1}{|\mathcal{Q}_A|}, \tag{12}$$

where $\sigma(\cdot)$ represents the standard deviation. The optimal threshold T_e is determined by selecting the candidate T_z that maximizes T_z' as follows:

$$T_e = \arg\max_{T_z}(T_z'). \tag{13}$$

The proposed threshold-setting mechanism ensures robust anomaly detection across diverse data and scenarios by leveraging a data-driven approach. By using K-means clustering with three initial states (i.e., normal, transitive, and anomalous), the mechanism adapts to varying distributions of anomaly probabilities. The initialization of cluster centers (i.e., 0, mean, and max values) provides a balanced separation of states, making the method flexible to datasets with different ranges and distributions of q_i values.

Algorithm 1 ART-FL

Require: multiplier r_k, learning rate $\bar{\eta}$, regularization parameter λ, decay coefficient Γ.

Ensure: Updated global model $\mathbf{w}_G$

1: Initialize: $h_k^p = h_k = 0$, $v_k = 0$, $r_k^t = \max\left\{1, \log\left(\bar{d}_k^t\right)\right\}$ where $\bar{d}_k^t = \frac{1}{t}\sum_{\tau=1}^{t} d_k^\tau$.
2: **[Procedure at Cloud Server]**
3: **for** $t = 1, 2, \ldots, T$ (global iterations) **do**
4: compute $\mathbf{w}_G^t$ $/* gettheupdateon \mathbf{w}_G^t */$ ▷ [Eq.(14)]
5: apply feature learning to updated $\mathbf{w}_G^t$ ▷ [Eq.(15)]
6: **end for**
7: **[Procedure of Local Client** k**]**
8: receive $\mathbf{w}_G^t$ from the server
9: Compute $\nabla s_k = \nabla f_k(\mathbf{w}_k^t) + \lambda(\mathbf{w}_k^t - \mathbf{w}_G^t)$
10: Set $h_k^p = h_k$
11: Update gradient $\nabla\zeta_k \leftarrow \nabla s_k - \nabla s_k^p + h_k^p$
12: Update local model $\mathbf{w}_k^{t+1} \leftarrow \mathbf{w}_k^t - r_k^t \bar{\eta} \nabla\zeta_k$
13: Update $h_k = \Gamma h_k + (1-\Gamma) v_k$
14: Compute $v_k = \nabla s_k(\mathbf{w}_G^t; \mathbf{w}_k^t)$
15: upload $\mathbf{w}_k^{t+1}$ to the server

E. GLOBAL MODEL AGGREGATION AND UPDATE

Up until now, we have explained the entire process of the anomaly detection model at the edge client. This model is then extended for use in an FL framework. We propose an asynchronous real-time FL, where the server updates the global model $\mathbf{w}_G$ as soon as it receives updates from any client, without waiting for the others. Each client maintains a local copy of $\mathbf{w}_G$, which may differ from those of other clients due to asynchronous updates. The server's task includes not only aggregating these updates but also performing feature learning to extract and enhance cross-client feature representations, thereby improving the overall model performance. Fig. 8 shows the update process of the ART-FL method. The server aggregates the updates from the clients as they arrive and conducts feature learning on the aggregated parameters to create a cross-client feature representation. The updated global model $\mathbf{w}_G$ is then sent to clients that are prepared for the next iteration. As clients collect new data samples, we implement a decay coefficient to balance the previous and current local gradients through a repetitive local computation process. Algorithm 1 depicts the pseudocode of the proposed ART-FL method. Learning on the cloud server is shown in lines 2-6 of the Algorithm 1, which involves aggregating and processing data received from the local clients. Learning on local clients is described in lines 7-15 of the Algorithm 1, where the focus is on updating local models based on individual client data and the aggregated information received from the cloud server. As shown in Fig. 8, when the server receives updates from slower clients (e.g., Client 2), it may have already performed multiple updates to the global model $\mathbf{w}_G$. This asynchronous updating issue, which occurs due to data heterogeneity and network delays, is typical in real-world scenarios. To cope with this, we develop a global feature representation on the server and implement a dynamic step size to train local clients.

1) LEARNING ON THE CLOUD SERVER

Each update cycle on the server begins by aggregating updates from client k as follows:

$$\begin{aligned}\mathbf{w}_G^{t+1} &= \mathbf{w}_G^t - \frac{n_k'}{N'}\left(\mathbf{w}_k^t - \mathbf{w}_k^{t+1}\right)\\ &= \mathbf{w}_G^t - \frac{n_k'}{N'}\left(\mathbf{w}_k^t - \left(\mathbf{w}_k^t - \eta_k^t \nabla\zeta_k\left(\mathbf{w}_G^t\right)\right)\right)\\ &= \mathbf{w}_G^t - \eta_k^t \frac{n_k'}{N'} \nabla\zeta_k\left(\mathbf{w}_G^t\right), \end{aligned} \tag{14}$$

where η_k^t is the learning rate of client k at iteration t, n_k' represents the number of new data received from client k, N' is the updated total number of samples across all clients, and $\nabla\zeta_k$ denotes the gradient of the loss function for client k. Following this aggregation, the server applies feature representation learning techniques to address any potential performance issues arising from asynchronous updates. We perform feature extraction on the first layer (e.g., GAT) to generate the feature representation and denote the parameters of this layer as $\mathbf{w}_{(1)}^{t+1}$. For each element $\mathbf{w}_{(1)}^{t+1}[i,j]$ in column $\mathbf{w}_{(1)}^{t+1}[j]$ of $\mathbf{w}_{(1)}^{t+1}$, the updated $\mathbf{w}_{(1)}^{t+1}$ is obtained as follows:

$$\mathbf{w}_{(1)}^{t+1}[i,j] = \alpha_{(1)}^{t+1}[i,j] \times \mathbf{w}_{(1)}^{t+1}[i,j], \tag{15}$$

where $\alpha_{(1)}^{t+1}[i,j]$ is computed through a softmax function over the absolute values of the weights as follows:

$$\alpha_{(1)}^{t+1}[i,j] \leftarrow \frac{\exp\left(\left|\mathbf{w}_{(1)}^{t+1}[i,j]\right|\right)}{\sum_j \exp\left(\left|\mathbf{w}_{(1)}^{t+1}[i,j]\right|\right)}. \tag{16}$$

2) LEARNING ON LOCAL CLIENTS

Local clients aim to align their models closely with the global model by addressing local deviations and incorporating regularization. The local objective function s_k, which penalizes large deviations between local and global models, is given by:

$$s_k(\mathbf{w}_k) = f_k(\mathbf{w}_k) + \frac{\lambda}{2}\|\mathbf{w}_k - \mathbf{w}_G\|^2, \tag{17}$$

where $f_k(\mathbf{w}_k)$ is the local loss function for client k and λ is the regularization parameter.

Given that the data continues arriving at local clients during the training process, each client is required to perform real-time learning. Each client retrieves the latest global model $\mathbf{w}_G^t$ from the cloud server and updates it with new local data. This requires balancing the impact of the previously learned parameters with the newly updated data using a decay coefficient Γ. At each global iteration t, client k receives the global model $\mathbf{w}_G^t$ from the server. Let ∇s_k^{p} denote the gradients computed during the previous update at client k. The optimization process at client k during this iteration involves adjusting the local model based on the new gradients and a

decay mechanism to blend historical gradient influences with current data as follows:

$$\nabla\zeta_k \leftarrow \nabla s_k - \nabla s_k^{\mathrm{p}} + h_k^{\mathrm{p}} \tag{18}$$

where ∇s_k is the gradient of the local objective function for client k and h_k^{p} is initialized to zero and updated each iteration to balance the old and new gradients using the decay coefficient Γ as follow:

$$h_k^{\mathrm{p}} = \Gamma h_k^{\mathrm{p}} + (1-\Gamma)\nabla s_k^{\mathrm{p}} \tag{19}$$

The learning rate η_k^t is employed to apply these adjustments to the local model. The local model $\mathbf{w}_k^{t+1}$ is then updated as follows:

$$\begin{aligned}\mathbf{w}_k^{t+1} &= \mathbf{w}_k^t - \eta_k^t \nabla\zeta_k(\mathbf{w}^t),\\ &= \mathbf{w}_k^t - \eta_k^t(\nabla f_k(\mathbf{w}_k^t) - \nabla s_k^{\mathrm{p}} + h_k^{\mathrm{p}} + \lambda(\mathbf{w}_k^t - \mathbf{w}^t)),\end{aligned} \tag{20}$$

where λ is a regularization parameter that helps mitigate overfitting by penalizing large deviations from the global model.

Variability in client update frequencies due to factors such as bandwidth, network delays, and data heterogeneity can affect model training. To cope with this, we implement a dynamic learning step size. If a client has a larger data volume or faces communication issues, its activation rate will be low, so we increase its learning step size accordingly. This dynamic adjustment is represented by r_k^t, which is calculated to reflect the activation frequency and network conditions of client k at iteration t. Initially, all clients start with a baseline learning rate $\eta_k^t = \bar{\eta}$. Thus, Eq. (20) for client k's model at iteration t is updated as follows:

$$\mathbf{w}_k^{t+1} = \mathbf{w}_k^t - r_k^t \bar{\eta} \nabla\zeta_k(\mathbf{w}_G^t), \tag{21}$$

where $r_k^t = \max\{1, \log(\bar{d}_k^t)\}$ and $\bar{d}_k^t = \frac{1}{t}\sum_{\tau=1}^{t} d_k^\tau$ is the average delay over the past t iterations. $\nabla\zeta_k$ includes contributions from the gradient of the loss function and any regularization terms, adjusted dynamically by r_k^t. This dynamic adjustment helps mitigate the impact of asynchronous updates based on past communication delays and varying data distributions across clients, ensuring more stable and efficient convergence. For a comprehensive proof of the convergence analysis, we refer the reader to Appendix .

VI. RESULTS

In this section, we describe the implementation details, present the findings, and conclude with a discussion.

A. IMPLEMENTATION DETAILS

We present a summary of our experimental setup, detailing the benchmark application used, the load generation and trace collection methods, the specifications for anomaly injection, and the parameter configurations and coding environment employed to implement the proposed solution.

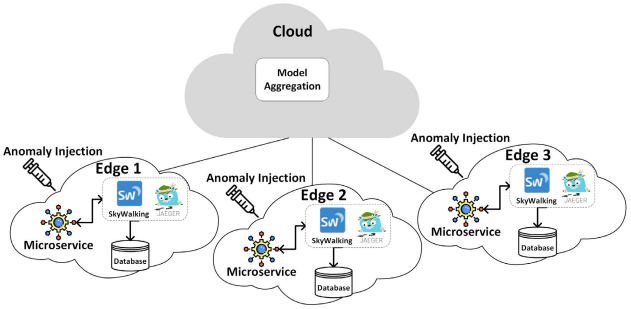


FIGURE 9. Lab setup for evaluation.

TABLE 3. Configuration parameters and settings for the lab setup.

Site	Parameter	Value
Cloud Server	Number of VMs	1 VM
	CPU	4 cores (core i7-8700)
	RAM	16 GB
	HDD	40G
Edge	Number of VMs	9 VMs
	CPU	4 cores for each VM
	RAM	8 GB for each worker VM 16 GB for master VM
	HDD	40 GB for each VM

1) BENCHMARK APPLICATION

In our evaluations, we utilized TrainTicket [1], a dynamic real-world benchmark designed specifically for microservices. TrainTicket simulates the core functions of a train ticket booking system, including ticket searches, reservations, payments, modifications, and user notifications. It adheres to microservice architecture principles and supports different interaction modes, such as synchronous and asynchronous calls, as well as message queues. The system consists of 41 business logic microservices (excluding database and infrastructure services) and is specifically developed to enable testing and experimentation with microservices and cloud-native technologies.

2) EXPERIMENTAL TESTBED

Fig. 9 depicts our lab setup, which consists of a lab testbed environment established within Ericsson Research's private cloud, also known as Xerces. Xerces is powered by an infrastructure that includes approximately 300 servers, all orchestrated by an Infrastructure-as-a-Service (IaaS) platform based on OpenStack. Our test setup consists of a Kubernetes cluster comprising four sub-clusters with a total of 10 Virtual Machines (VMs) operating on Ubuntu 20.04. The central cluster houses one VM designated as the Kubernetes main node. In each of the three edge clusters, two VMs are serving as Kubernetes worker nodes, along with a main node. These virtual machines' specifications are listed in Table 3. For monitoring and trace gathering within these Kubernetes clusters, we utilized Jaeger.[2] Additionally,

[2]https://www.jaegertracing.io/

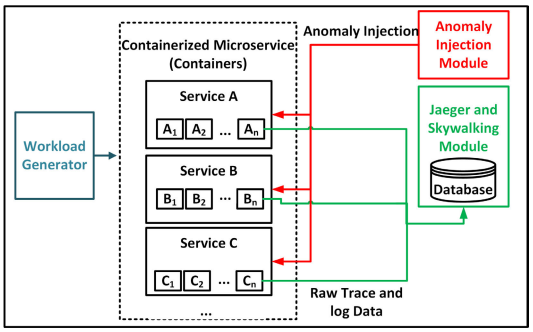


FIGURE 10. Load generation and data collection from the testbed.

Apache SkyWalking[3] is employed as our distributed tracing platform for log collection. Each cluster is equipped with one instance of both Jaeger and SkyWalking. The TrainTicket application is deployed on all main nodes. Kubernetes Cluster Federation (Kubefed) is used to streamline application deployment processes, such as setting up Jaeger and SkyWalking services across various clusters. Kubefed offers a centralized approach to manage and configure multiple Kubernetes clusters through a single API set, operating from the central site in our configuration.

3) LOAD GENERATION AND DATA COLLECTION

Fig. 10 illustrates our approach for generating workloads and collecting data from Kubernetes clusters. As explained in Section VI-A.2, we deploy a container-based application on three Kubernetes clusters. In Fig. 10, a container is defined as a grouping of one or multiple containers forming a complete microservice, aligning with the Kubernetes concept of a pod. To enhance the diversity of collected data, we incorporate an anomaly injection module, injecting various anomalies into the containers. Each Kubernetes cluster comprises multiple nodes (hosts), with each node accommodating several containers. In our setup, each container hosts a specific microservice, and a microservice can be concurrently deployed across multiple containers. We employ a distributed tracing module, utilizing Jaeger and SkyWalking, on every VM to trace request flows within microservices. A workload generator is deployed to emulate real user requests. We simulate a range of user behaviors to generate realistic workloads, from basic tasks like visiting the homepage and searching for train schedules, to more complex actions such as user authentication and purchasing tickets. Furthermore, we implement a dynamic scaling mechanism for the simulated user base, tailoring the frequency of each user behavior over time. This methodology facilitates the creation of a dynamic, varied mix of requests, effectively simulating real-life web traffic. For the generation of these synthetic workloads, we deploy the Performance Testing-based Application Monitoring (PPTAM) framework [43], which delineates five unique user profiles. Minor adjustments were made to PPTAM to allow for real-time variation in the user distribution across different types of requests. The generation of workload was evenly distributed across various request categories to ensure comprehensive coverage of all microservice benchmarks, as illustrated in Fig. 10. From this setup, we successfully gathered 189,486 execution traces within the microservice architecture.

[3]https://skywalking.apache.org/

TABLE 4. List of injected anomalies into trainticket.

Object	Anomaly Injection Operation
Physical	CPU Overload: Inject a sudden spike in CPU usage in a VM. Memory Leak: Gradually consume more memory in a VM to mimic a memory leak. Network Congestion: Introduce network congestion by limiting bandwidth.
Database	Slow Query Processing: Drop indexes from some tables. Deadlock Problem: Connections lock tables, causing deadlocks. Data Flushing Fault: Truncate a table, slowing down insertions due to a storage engine bug.
Application	Unauthorized Access: Incorrect password for database access. Access Configuration Error: Incorrect user access authentication. Service Access Failure: Incorrect path set for microservices. Service Invocation Failure: Error in service implementation leading to inaccurate responses.
Pod	Pause Container: Use Docker's "PAUSE" command on a container. Error Hang: Trigger an error hang in a microservice container. CPU Load Increase: In a container, increase CPU load to 100%. Disable Pod: Randomly disable a specific pod.

4) ANOMALY INJECTION

We implemented an anomaly injector with configurable options for selecting injection targets, anomaly types, injection timing, duration, and intensity. This injector is designed to be packaged as a file-system layer within microservice containers, enabling remote activation during the training phase. It includes different types of anomalies that have the potential to violate the SLOs, as shown in Table 4. Every service instance receives uniform random injections of anomalies of various kinds with tunable injection timing and intensity. We first chose a physical resource, database, application, and pod randomly and then applied the corresponding injection strategy randomly to the microservice. Every anomaly type has an injection intensity randomly chosen from the range [0, 1]. Each anomaly injection in our experimentation has a duration of 3 minutes. To minimize the potential cross-influence of different injection operations, we ensure that the interval between successive injections is greater than 6 minutes, while the anomaly type and intensity are chosen randomly. 28.7% of the entire trace data consists of anomalous traces.

5) PARAMETER SETTING AND CODING ENVIRONMENT

The proposed MS-FLAD method was developed with PyTorch 1.10 and Python 3.10.9, employing PyTorch Geometric 2.2 for the GAT and adapting the nnPU algorithm [41] for PU learning. We incorporated semantic analysis using the pre-trained BERT-Base model [38] and WordPiece tokenizer [37], with time features embedded into 100 dimensions. The MS-FLAD method features a three-layer GAT with the first two layers using three attention heads and the final layer using one. Batch normalization followed each layer, and training parameters included a batch size of 128, a prior probability π_p of positive data of 0.15, and hyperparameter β set to 0. The training ran for 50 epochs with an

Adam [42] optimizer learning rate of 0.001. Data was divided into training, validation, and testing sets in a 3:1:6 ratio, with $\sim$ 10% of anomalous traces in the training set labeled as positive, making up about 2% of the training data. To evaluate the detection performance, we also conduct training using a centralized variant of our proposed approach called Microservice-Anomaly Detection (MS-AD), where all the data is processed centrally without the constraints of data distribution and privacy concerns. As for the centralized MS-AD implementation, we utilize the complete original dataset for training on the cloud server in our testbed.

6) EVALUATION METRICS

We consider precision, recall, and F_1-score to evaluate the performance of different anomaly detection methods under study. A large value of precision, recall, and F_1-score indicates that the model is efficient at flagging true anomalies while minimizing false alarms and missed anomalies, which is key for anomaly detection to meet the accuracy requirement. Moreover, time efficiency is measured using detection time, which is the time required to process a single trace of data, including the time needed for anomaly detection. A short detection time is crucial to meet the real-time and time-efficiency requirements of the anomaly detection system. Training time refers to the total time taken to train the model across all iterations until convergence. Moreover, scalability is evaluated in terms of how the response time of our anomaly detection model changes as trace size (i.e., number of events) increases. Finally, resource overhead is an important metric for ensuring cost-efficient communication and computation between edge clients and the cloud server. Communication overhead measures the amount of data exchanged between clients and the cloud server, while computation overhead refers to the processing time consumed by the clients to locally train the model before synchronization.

B. EXPERIMENTAL RESULTS

In this section, we present our evaluation results, which are categorized into two main sections. The first section presents the performance of anomaly detection in terms of effectiveness, time efficiency, communication overhead, and scalability. The second section examines the impact of parameter configuration on performance, along with an ablation study.

1) ANOMALY DETECTION RESULTS

a: EFFECTIVENESS

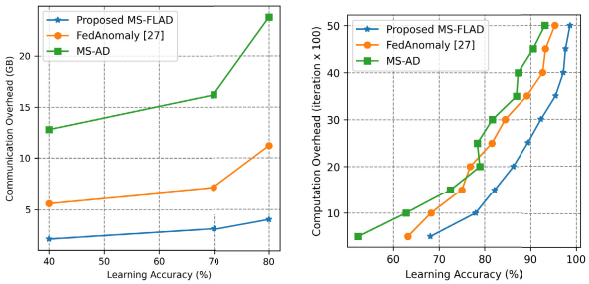
We compare our proposed MS-FLAD anomaly detection algorithm with existing works, including Microscope [14], TraceAnomaly [12], MultimodalTrace [13], TDAD [8], Seer [9], LogAnomaly [20], FedAnomaly [27], and FL-SmartGrid-AD [29]. Among these methods, FedAnomaly [27] and FL-SmartGrid-AD [29] are FL-based models, whereas the others are centralized methods, which are compared to the centralized variant MS-AD of our proposed method. Microscope [14], TraceAnomaly [12], MultimodalTrace [13], TDAD [8] rely on tracing data, LogAnomaly [20] only relies on log data and FedAnomaly [27], and FL-SmartGrid-AD [29] rely on metric data for anomaly detection. Table 5 illustrates the obtained results of recall, precision, and F_1-score for different anomaly detection solutions. The proposed MS-FLAD achieves a recall of 98.7%, precision of 94%, and an F_1-score of 96.3%. It is evident that our proposed MS-FLAD and MS-AD outperform other methods that depend solely on a single data source and utilize a centralized approach. This is because incorporating log messages and trace data in MS-FLAD, representing intra-service and inter-service behaviors, respectively, enables more timely and accurate detection of anomalies. Moreover, the proposed MS-FLAD and MS-AD outperform Seer [9], even though they utilized multiple source data (trace and metric). While Seer [9] focuses on detecting response time and invocation path anomalies in a unified manner, it does not consider the graph-based structure of traces and anomalies in resource usage. Also, while LogAnomaly [20] aims to detect sequential anomalies of log templates, it is outperformed by the proposed method. Our proposed MS-FLAD outperforms in precision, recall, and the F_1-score when compared to TDAD [8], Microscope [14], TraceAnomaly [12], and MultimodalTrace [13]. These methods are based on sequence-based trace representation, which is not suitable for capturing the causal connections among spans. The TraceAnomaly [12] only takes into account the response time and service sequences seen in traces. MultimodalTrace [13] only uses the operation sequences of traces to train an LSTM-based model. Additionally, these trace-based anomaly detection methods do not account for intra-service behavior. In contrast, our proposed MS-FLAD method considers the intra-service behavior of service instances. Therefore, MS-FLAD outperforms FedAnomaly [27] and FL-SmartGrid-AD [29] by achieving up to a 96.3% F_1-score. This is also because MS-FLAD incorporates GNN and PU learning to inherit the anomaly detection benefits of SCG.

TABLE 5. Performance comparison for anomaly detection.

Approaches	Effectiveness(%)			Time Efficiency	
	Recall	Precision	F_1-score	Training time (m)	Detection time (ms)
Microscope [14]	39.6	94.3	55.7	450	4.7
TraceAnomaly [12]	65.9	58.0	61.5	30	137
Multimodal Trace [13]	60.1	96.0	73.8	30.4	148
TDAD [8]	87.1	93.0	89.8	35	0.1
Seer [9]	67.8	93.8	78.7	32	98
LogAnomaly [20]	62.7	98.5	76.6	77.6	27.8
FedAnomaly [27]	99.9	86.4	92.6	890	12
FL-SmartGrid [29]	96.2	87.5	91.7	82.1	2.2
MS-AD	95.0	87.1	90.9	67.1	0.7
Proposed MS-FLAD	98.7	94.0	96.3	21	0.4

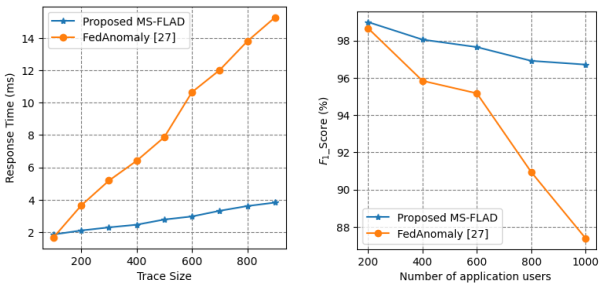
b: TIME-EFFICIENCY

Table 5 illustrates the training time and detection time for various methods under consideration. We observe from the



(a) (b)

FIGURE 11. (a) Communication overhead vs. learning accuracy and (b) learning accuracy vs. computation overhead.



(a) (b)

FIGURE 12. Scalability of the proposed MS-FLAD and FedAnomaly, in terms of (a) response time of prediction vs. trace size, (b) F_1-score vs. the number of application users.

table that our proposed MS-FLAD method, in comparison to trace-based methods such as TraceAnomaly [12], MultimodalTrace [13], shows a slower pace in model training, though it outperforms other methods in terms of detection time. Also, MS-FLAD is notably faster than all compared approaches in terms of detection time. We also observe from the table that the centralized variant MS-AD of our proposed method achieves a higher training time compared to other trace-based methods, but a lower detection time. This happens because the MS-AD method considers logs as well as traces, which incurs additional training time. This efficiency is attributed to the anomaly detection model being fully characterized by its network parameters θ, eliminating the need to store data for anomaly detection. Given that MS-FLAD is an FL approach, it is faster in training than the centralized approaches. Also, since the proposed MS-FLAD allows models to be trained locally, detection can occur in real time at the edge, where data is generated. This, as a result, minimizes the time it takes to detect anomalies. LogAnomaly [20], which is a log-based method, splits the event sequences into several subsequences for testing and training by using sliding windows. Each subsequence in this method utilizes count vector sequences, the dimensions of which are based on the number of log events. With our dataset containing over 800 log and span events, this approach encounters the curse of dimensionality. This issue is common in microservice systems with a large number of log or span events, rendering traditional log anomaly detection methods less effective in such environments. FL-SmartGrid-AD [29] is slower in training and testing than our proposed MS-FLAD while using a simpler sequence representation for traces and logs. This is due to the fact that GNNs treat each event in the graph as a network unit, allowing for concurrent message passing, while GRUs in FL-SmartGrid-AD [29] process each event in a long sequence serially, resulting in slower performance. Given that the FL approach shares computational resources with clients, the total time is notably reduced, even when factoring in the additional time required for FL averaging. Multiple computing instances from various clients sharing computational resources and parallel computing contribute to a reduction in the overall training time required to achieve optimal performance.

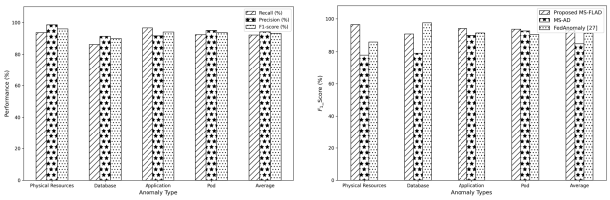
c: RESOURCE OVERHEAD

Fig. 11(a) compares the communication overhead in gigabytes (GB) between MS-AD, FedAnomaly [27], and the proposed MS-FLAD at three distinct learning accuracy levels. MS-FLAD consistently maintains the lowest communication overhead, reducing it by approximately 3x to 4x compared to MS-AD, demonstrating its superior efficiency in minimizing communication costs while achieving high accuracy. Fig. 11(b) illustrates the comparison of the proposed MS-FLAD algorithm, FedAnomaly [27], and MS-AD in terms of learning accuracy as a function of computation overhead (iteration). The proposed MS-FLAD achieves the highest accuracy with faster convergence compared to the other methods.

d: SCALABILITY

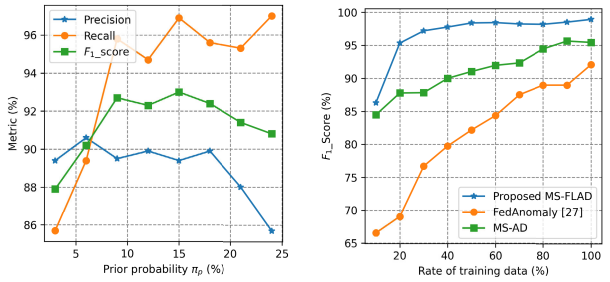
Next, we examine how response time is affected by trace size, which is the total number of span and log events in the trace. Fig. 12(a) depicts trace size vs. response time. In our evaluations, we divided the range of event numbers from 0 to 900 into nine equal-size distinct groups, each spanning 100 events, and then randomly selected 100 traces from each group. For each trace, the anomaly detection model was run 300 times to determine the average response time for making a prediction. The evaluation focuses on the comparison of average response times between MS-FLAD and FedAnomaly [27] across traces of different sizes. This comparison was chosen due to the unique performance characteristics of each method, as previously detailed. Specifically, FedAnomaly [27] outperforms in detection accuracy, achieving a higher F_1-score than competing methods. Fig. 12(a) shows a linear increase in response time with trace size for both methods. Significantly, MS-FLAD maintains quicker response times in comparison to FedAnomaly [27].

Moreover, we evaluate the proposed MS-FLAD scalability under varying numbers of application users. We systematically increased the number of users from 200 to 1000 to simulate diverse scenarios of user engagement. As shown in Fig. 12(b), the proposed MS-FLAD consistently maintained the F_1-score above 95%, even with the growing number of application users. This robust performance illustrates the method's reliability and scalability. When compared



(a) (b)

FIGURE 13. Performance evaluation: (a) Recall, precision, and F_1-score of the proposed MS-FLAD across different anomaly types. (b) F_1-score comparison across different anomaly types for MS-FLAD, MS-AD, and FedAnomaly [27].



(a) (b)

FIGURE 14. (a) Impact of prior probability, π_p, on the proposed MS-FLAD performance, (b) F_1-score vs. different training data rate.

to the FedAnomaly [27] approach, our proposed MS-FLAD demonstrates superior performance across all user group sizes. Particularly, the gap in F_1-score between the two methods becomes more noticeable as the number of users increases.

2) IMPACT OF CONFIGURATION PARAMETERS ON THE PERFORMANCE

The experimental results outlined in Fig 13(a) illustrate the effectiveness of our approach in detecting anomalies with high precision, recall, and F_1-score across various anomaly types. The results are categorized into four groups, namely anomalies related to physical resources, databases, applications, and pods. Furthermore, we conduct a comparative analysis against MS-AD and FedAnomaly [27], utilizing the F_1-score as a comprehensive metric capturing both precision and recall as shown in Fig. 13(b). The average F_1-score across all anomaly types is presented. Our proposed MS-FLAD outperforms FedAnomaly [27] by 2% and outperforms MS-AD by 7%.

Fig. 14(a) illustrates performance vs. the prior probability π_p of anomalous traces as described in Eq. (7). The prior probability π_p represents an estimation of the proportion of anomalous traces within the dataset, which directly affects the effectiveness of the nnPU risk estimator during training. As shown in Fig. 14(a), we can observe the influence of π_p on the performance of MS-FLAD, including precision, recall, and F_1-score. Generally, as π_p increases, recall tends to rise while precision decreases. This behavior is attributed to the model's inclination to predict more traces as anomalous with higher values of π_p. When π_p is approximately 15%, which closely matches the actual proportion of anomalous traces in the dataset (15.4%), MS-FLAD achieves its highest F_1-score. Furthermore, MS-FLAD consistently maintains high performance, with F_1-scores close to or exceeding 0.9, when π_p falls within the range of 2.5% to 25%. Consequently, MS-FLAD demonstrates strong performance when π_p falls within a reasonable range, with its optimal performance occurring when π_p aligns closely with the actual proportion of anomalous traces in the dataset.

TABLE 6. Ablation performance of MS-FLAD with different FL approaches.

MS-FLAD Variant	Recall (%)	Precision (%)	F_1-score (%)	Training Time (m)
with FedAvg [16]	55	87	66	45
with FedProx [45]	65	84	72	26
with proposed ART-FL	98.7	94	96.3	21

To evaluate the incremental real-time learning process, we examined how the performance changes as the amount of training data grows. This is illustrated in Fig. 14(b), where we conducted experiments using different proportions of the clients' training data. The results show that ART-FL consistently performs the best as more data becomes available. In contrast, FedAnomaly [27] shows significant fluctuations in performance due to the instability of synchronous approaches when local data increases. Compared to the MS-AD model, our asynchronous ART-FL approach maintains more stable performance as data grows.

3) ABLATION STUDY

We perform an ablation study to assess the effects of different FL approaches on the model. We compare the proposed MS-FLAD with both synchronous and asynchronous FL approaches. The synchronous model, As shown in Table 6, the synchronous model, FedAvg, aggregates updates using a simple averaging method [16], resulting in a training time of 45. Despite this slower training, it achieves a recall of 55%, precision of 87%, and an F_1-score of 66%, indicating its challenges in efficiently managing distributed data. Conversely, the asynchronous model, FedProx [44], employs a weighted average to accommodate updates as they arrive, leading to a training time of 26. This model demonstrates better performance with a recall of 65%, precision of 84%, and an F_1-score of 72%, but still underperforms our ART-FL model. Lastly, our ART-FL model demonstrates a processing time of 21. It significantly enhances both speed and accuracy with a recall of 98.7%, precision of 94%, and an F_1-score of 96.3%, making it a competitive alternative that nearly rivals the performance of the global model.

C. DISCUSSIONS

The proposed MS-FLAD addresses the key limitations of centralized and FL methods by enhancing scalability,

resource utilization, and anomaly detection performance while preserving data privacy. Centralized methods rely on sharing raw data, which poses privacy risks and limits its scalability. MS-FLAD eliminates this dependency by processing data locally, achieving superior anomaly detection accuracy without compromising privacy. Traditional FL approaches like FedAvg [16] and FedProx [44] require synchronous updates, leading to delays in heterogeneous environments. MS-FLAD's ART-FL model leverages asynchronous updates, reducing idle time and enabling real-time anomaly detection. Quantitatively, ART-FL achieves up to 46% higher accuracy in terms of F_1-score and reduces training time by up to 2.1x compared to FedAvg and FedProx. Compared to FedAnomaly [27], MS-FLAD achieves up to 4% higher anomaly detection accuracy and reduces communication overhead by 3x to 4x while maintaining faster detection times across varying trace sizes.

While this work focuses primarily on computational efficiency, it inherits the fundamental privacy-preserving properties of FL by keeping all training data localized. To further enhance data security, future iterations could integrate advanced privacy-preserving mechanisms, such as differential privacy or secure multi-party computation. For example, adding noise to local updates or encrypting communication between edge clients and the cloud server could provide protection against potential data leakage.

Despite its strengths, ART-FL has several limitations. First, its scalability in extremely large networks with thousands of clients or highly heterogeneous data distributions requires further validation, as communication costs and computational overhead may become bottlenecks. Second, while ART-FL preserves data locality, it does not integrate advanced privacy-preserving mechanisms, which could enhance its applicability in sensitive domains.

VII. CONCLUSION

In this paper, we proposed an asynchronous FL method called ART-FL for detecting anomalies in trace and log data of microservice cloud applications. ART-FL enhances computational efficiency by allowing the cloud server to aggregate updates without waiting for slower clients. Our proposed method for edge clients combines GNN and PU learning for the accurate detection of anomalous traces. The training process in our proposed method involves the use of a small number of labeled anomalous traces as well as a relatively large number of unlabeled traces. Our trace-driven evaluations on a microservice benchmark demonstrate that the proposed method outperforms the existing anomaly detection methods by 4% in terms of F_1-score and by 5x in terms of detection time. Compared to other asynchronous methods, ART-FL delivers superior performance, particularly in managing local streaming data. Time efficiency assessments indicate that ART-FL operates faster than traditional synchronized FL approaches. A promising direction for future work is to evaluate the proposed approach across diverse microservice architectures. Also, future research will explore the integration of advanced privacy-preserving techniques within the ART-FL framework to enhance data security. Finally, developing a unified method that combines microservice application logs, resource metrics, and trace data holds the potential to address the challenges of anomaly detection in large-scale and complex microservice systems.

APPENDIX
CONVERGENCE ANALYSIS

In this section, we present the theoretical analysis of the convergence of ART-FL. We first define the key terms and assumptions.

A. DEFINITIONS

Definition 1 (Smoothness): The function f has Lipschitz continuous gradients with constant $L > 0$ (i.e., f is L-smooth) if $\forall x_1, x_2$,

$$f(x_1) - f(x_2) \leq \langle \nabla f(x_2), x_1 - x_2 \rangle + \frac{L}{2}\|x_1 - x_2\|^2. \quad (22)$$

Definition 2 (Strong convexity): The function f is μ-strongly convex with $\mu > 0$ if $\forall x_1, x_2$,

$$f(x_1) - f(x_2) \geq \langle \nabla f(x_2), x_1 - x_2 \rangle + \frac{\mu}{2}\|x_1 - x_2\|^2. \quad (23)$$

Definition 3 (Bounded gradient dissimilarity): The local functions f_k are V-locally dissimilar at $\mathbf{w}_G$ if:

$$\mathbb{E}\|\nabla f_k(\mathbf{w}_k)\|^2 \leq \|\nabla F(\mathbf{w}_G)\|^2 V^2. \quad (24)$$

Here, $V(\mathbf{w}_G)$ is defined as $\sqrt{\mathbb{E}\|\nabla F(\mathbf{w}_G)\|^2 / \|\nabla f_k(\mathbf{w}_k)\|^2}$ when $\|\nabla f_k(\mathbf{w}_k)\|^2 \neq 0$.

B. ASSUMPTIONS

Assumption 1: Suppose the following hold:

1) The central objective function $F(\mathbf{w}_G)$ is bounded from below, i.e., $F_{\min} = F(\mathbf{w}_G^*) > -\infty$.
2) There exists $\varepsilon > 0$ such that $\mathbb{E}(\nabla f_k(\mathbf{w}_k)) \leq \|\nabla F(\mathbf{w}_G)\|$, and $\nabla F(\mathbf{w}_G)^\top \mathbb{E}(\nabla f_k(\mathbf{w}_k)) \geq \varepsilon \|\nabla F(\mathbf{w}_G)\|^2$ holds for all $\mathbf{w}_G$.

Lemma 1: If $F(\mathbf{w}_G)$ is μ-strongly convex, then under Assumption 1.1:

$$2\mu(F(\mathbf{w}_G^t) - F(\mathbf{w}_G^*)) \leq \|\nabla F(\mathbf{w}_G^t)\|^2. \quad (25)$$

C. THEOREMS AND PROOFS

Theorem 1: Suppose the central objective function $F(\mathbf{w}_G)$ is L-smooth and μ-strongly convex. Assume the local functions f_k are bounded dissimilar. Let Assumption 1 hold. Let $\bar{\eta}_k \leq \eta_t^k < \eta_k = \frac{2\varepsilon N'}{LV^2 n_k'}$, then after T global updates on the server, ART-FL converges to a global optimum $\mathbf{w}_G^*$:

$$\mathbb{E}[F(\mathbf{w}_G^T) - F(\mathbf{w}_G^*)] \leq (1 - 2\mu\gamma'\bar{\eta}_k)^T (F(\mathbf{w}_G^0) - F(\mathbf{w}_G^*)). \quad (26)$$

where $\gamma' = \varepsilon - \frac{L\eta_k V^2}{2}$.

Theorem 2: Suppose the central objective function $F(\mathbf{w}_G)$ is L-smooth and non-convex. Let Assumption 1 hold. Assume

the local functions f_k are bounded dissimilar. If it holds that $\eta_t^k < \frac{2\varepsilon-1}{LV^2} \leq \max(r_t^k \bar{\eta}) = \bar{\eta}$ for all t, then after T global iterations:

$$\sum_{t=0}^{T-1} \eta_t^k \mathbb{E}[\|\nabla F(\mathbf{w}_G^t)\|^2] \leq F(\mathbf{w}_G^0) - F(\mathbf{w}_G^*). \tag{27}$$

The detailed proofs of Theorems 1 and 2 are provided in the respective appendices.

D. PROOF OF THEOREM 1

Using the L-smooth property of $F(w)$:

$$F(w^{t+1}) - F(w^t) \leq \langle \nabla F(w^t), w^{t+1} - w^t \rangle + \frac{L}{2}\|w^{t+1} - w^t\|^2. \tag{28}$$

Substituting the update rule of ART-FL and simplifying under the given assumptions, we derive:

$$\mathbb{E}[F(w_{t+1}) - F(w^t)] \leq -\eta_k \epsilon \|\nabla F(w^t)\|^2 + \frac{L\eta_k^2 V^2}{2}\|\nabla F(w^t)\|^2. \tag{29}$$

Letting $\gamma = \epsilon - \frac{L\eta_k V^2}{2}$, we obtain:

$$\mathbb{E}[F(w_{t+1}) - F(w^*)] \leq (1 - 2\mu\gamma\eta_k)[F(w_t) - F(w^*)]. \tag{30}$$

By telescoping this inequality over T iterations, we complete the proof.

E. PROOF OF THEOREM 2

Using similar steps for the non-convex case, we sum up the expected gradient norm over T iterations:

$$\sum_{t=0}^{T-1} \eta_k \mathbb{E}[\|\nabla F(w_t)\|^2] \leq F(w_0) - F(w^*). \tag{31}$$

Scaling the learning rate with $\frac{1}{2}$, we obtain the final bound:

$$\sum_{t=0}^{T-1} \frac{\eta_k}{2} \mathbb{E}[\|\nabla F(w_t)\|^2] \leq F(w_0) - F(w^*). \tag{32}$$

The convergence rate of ART-FL is affected by the bounded gradient dissimilarity V, the learning rate η_k, and the smoothness constant L. These results hold for both convex and non-convex central objectives.

REFERENCES

[1] X. Zhou et al., "Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study," *IEEE Trans. Softw. Eng.*, vol. 47, no. 2, pp. 243–260, Feb. 2021.

[2] E. Samikwa, A. Di Maio, and T. Braun, "DFL: Dynamic federated split learning in heterogeneous IoT," *IEEE Trans. Mach. Learn. Commun. Netw.*, vol. 2, pp. 733–752, 2024.

[3] N. Zhao et al., "Identifying bad software changes via multimodal anomaly detection for online service systems," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Aug. 2021, pp. 527–539.

[4] P. Chen, Y. Qi, and D. Hou, "CauseInfer: Automated end-to-end performance diagnosis with hierarchical causality graph in cloud environment," *IEEE Trans. Services Comput.*, vol. 12, no. 2, pp. 214–230, Mar. 2019.

[5] M. Jin et al., "An anomaly detection algorithm for microservice architecture based on robust principal component analysis," *IEEE Access*, vol. 8, pp. 226397–226408, 2020.

[6] M. Raeiszadeh, A. Ebrahimzadeh, R. H. Glitho, J. Eker, and R. A. F. Mini, "Real-time adaptive anomaly detection in industrial IoT environments," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 6, pp. 6839–6856, Dec. 2024.

[7] T. Wang, W. Zhang, J. Xu, and Z. Gu, "Workflow-aware automatic fault diagnosis for microservice-based applications with statistics," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 4, pp. 2350–2363, Dec. 2020.

[8] M. Raeiszadeh, A. Ebrahimzadeh, A. Saleem, R. H. Glitho, J. Eker, and R. A. F. Mini, "Real-time anomaly detection using distributed tracing in microservice cloud applications," in *Proc. IEEE 12th Int. Conf. Cloud Netw. (CloudNet)*, Nov. 2023, pp. 36–44.

[9] Y. Gan et al., "Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices," in *Proc. ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2019, pp. 19–33.

[10] S. Nedelkoski, J. Cardoso, and O. Kao, "Anomaly detection and classification using distributed tracing and deep learning," in *Proc. 19th IEEE/ACM Int. Symp. Cluster, Cloud Grid Comput. (CCGRID)*, May 2019, pp. 241–250.

[11] J. Bogatinovski, S. Nedelkoski, J. Cardoso, and O. Kao, "Self-supervised anomaly detection from distributed traces," in *Proc. IEEE/ACM 13th Int. Conf. Utility Cloud Comput. (UCC)*, Dec. 2020, pp. 342–347.

[12] P. Liu et al., "Unsupervised detection of microservice trace anomalies through service-level deep Bayesian networks," in *Proc. IEEE 31st Int. Symp. Softw. Rel. Eng. (ISSRE)*, Oct. 2020, pp. 48–58.

[13] S. Nedelkoski, J. Cardoso, and O. Kao, "Anomaly detection from system tracing data using multimodal deep learning," in *Proc. IEEE 12th Int. Conf. Cloud Comput. (CLOUD)*, Jul. 2019, pp. 179–186.

[14] J. Lin, P. Chen, and Z. Zheng, "Microscope: Pinpoint performance issues with causal graphs in micro-service environments," in *Proc. Int. Conf. Serv. Orient. Comput.*, 2018, pp. 3–20.

[15] P. He, J. Zhu, S. He, J. Li, and M. R. Lyu, "Towards automated log parsing for large-scale log data analysis," *IEEE Trans. Depend. Sec. Comput.*, vol. 15, no. 6, pp. 931–944, Nov. 2018.

[16] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, 2017, pp. 1273–1282.

[17] B. Li et al., "Enjoy your observability: An industrial survey of microservice tracing and analysis," *Empirical Softw. Eng.*, vol. 27, no. 1, pp. 1–28, Jan. 2022.

[18] T. Davidson, E. Wall, and J. Mace, "A qualitative interview study of distributed tracing visualisation: A characterisation of challenges and opportunities," *IEEE Trans. Vis. Comput. Graphics*, vol. 30, no. 7, pp. 3828–3840, Jul. 2023.

[19] Y. Guo, Y. Wu, Y. Zhu, B. Yang, and C. Han, "Anomaly detection using distributed log data: A lightweight federated learning approach," in *Proc. IEEE Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2021, pp. 1–8.

[20] W. Meng et al., "Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, vol. 19, 2019, pp. 4739–4745.

[21] X. Zhang et al., "Robust log-based anomaly detection on unstable log data," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Aug. 2019, pp. 807–817.

[22] T. D. Nguyen, S. Marchal, M. Miettinen, H. Fereidooni, N. Asokan, and A.-R. Sadeghi, "DÏoT: A federated self-learning anomaly detection system for IoT," in *Proc. IEEE Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Jul. 2019, pp. 756–767.

[23] R. A. Sater and A. B. Hamza, "A federated learning approach to anomaly detection in smart buildings," *ACM Trans. Internet Things*, vol. 2, no. 4, pp. 1–23, Nov. 2021.

[24] V. Mothukuri, P. Khare, R. M. Parizi, S. Pouriyeh, A. Dehghantanha, and G. Srivastava, "Federated-learning-based anomaly detection for IoT security attacks," *IEEE Internet Things J.*, vol. 9, no. 4, pp. 2545–2554, Feb. 2021.

[25] H. T. Truong et al., "Light-weight federated learning-based anomaly detection for time-series data in industrial control systems," *Comput. Ind.*, vol. 140, Sep. 2022, Art. no. 103692.

[26] B. Li, Y. Wu, J. Song, R. Lu, T. Li, and L. Zhao, "DeepFed: Federated deep learning for intrusion detection in industrial cyber–physical systems," *IEEE Trans. Ind. Informat.*, vol. 17, no. 8, pp. 5615–5624, Aug. 2021.

[27] S. Ma et al., "Privacy-preserving anomaly detection in cloud manufacturing via federated transformer," *IEEE Trans. Ind. Informat.*, vol. 18, no. 12, pp. 8977–8987, Dec. 2022.
[28] B. Li, S. Ma, R. Deng, K. R. Choo, and J. Yang, "Federated anomaly detection on system logs for the Internet of Things: A customizable and communication-efficient approach," *IEEE Trans. Netw. Serv. Manage.*, vol. 19, no. 2, pp. 1705–1716, Jun. 2022.
[29] J. Jithish, B. Alangot, N. Mahalingam, and K. S. Yeo, "Distributed anomaly detection in smart grids: A federated learning-based approach," *IEEE Access*, vol. 11, pp. 7157–7179, 2023.
[30] Y. Liu et al., "Deep anomaly detection for time-series data in industrial IoT: A communication-efficient on-device federated learning approach," *IEEE Internet Things J.*, vol. 8, no. 8, pp. 6348–6358, Apr. 2021.
[31] T. T. Huong et al., "Detecting cyberattacks using anomaly detection in industrial control systems: A federated learning approach," *Comput. Ind.*, vol. 132, Nov. 2021, Art. no. 103509.
[32] L. Cui et al., "Security and privacy-enhanced federated learning for anomaly detection in IoT infrastructures," *IEEE Trans. Ind. Informat.*, vol. 18, no. 5, pp. 3492–3500, May 2022.
[33] X. Kong et al., "Federated graph anomaly detection via contrastive self-supervised learning," *IEEE Trans. Neural Netw. Learn. Syst.*, early access, Jun. 20, 2024, doi: 10.1109/TNNLS.2024.3414326.
[34] L. Meng, F. Ji, Y. Sun, and T. Wang, "Detecting anomalies in microservices with execution trace comparison," *Future Gener. Comput. Syst.*, vol. 116, pp. 291–301, Mar. 2021.
[35] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, Jun. 2017, pp. 33–40.
[36] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manage.*, vol. 24, no. 5, pp. 513–523, Jan. 1988.
[37] M. Schuster and K. Nakajima, "Japanese and Korean voice search," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Mar. 2012, pp. 5149–5152.
[38] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in *Proc. Conf. Empirical Methods Natural Lang. Syst. Demonstrations*, 2020, pp. 38–45.
[39] R. Fielding and J. Reschke, *Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content*, document RFC 7231, 2014.
[40] P. Velickovic et al., "Graph attention networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017, pp. 48510–48550.
[41] R. Kiryo, G. Niu, M. C. D. Plessis, and M. Sugiyama, "Positive-unlabeled learning with non-negative risk estimator," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1675–1685.
[42] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015, pp. 1–15.
[43] A. Avritzer et al., "PPTAM: Production and performance testing based application monitoring," in *Proc. Companion ACM/SPEC Int. Conf. Perform. Eng.*, Mar. 2019, pp. 39–40.
[44] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Mach. Learn. Syst.*, vol. 2, 2020, pp. 429–450.



MAHSA RAEISZADEH received the B.Sc. degree in IT engineering from Shiraz University of Technology, in 2014, and the M.Sc. degree in IT engineering from Shiraz University, Shiraz, Iran, in 2018. She is currently pursuing the Ph.D. degree with the Telecommunication Service Engineering Research Laboratory, Concordia University, Montreal, QC, Canada. From 2018 to 2020, she was with Shiraz University. She is a member with the Telecommunication Service Engineering Research Laboratory, Concordia University. Her research interests include cloud computing, the IoT, machine learning, and anomaly detection.



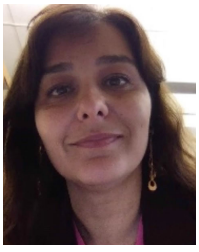
AMIN EBRAHIMZADEH received the B.Sc. and M.Sc. degrees in electrical engineering from the University of Tabriz, Tabriz, Iran, in 2009 and 2011, respectively, and the Ph.D. degree (Hons.) in telecommunications from the Institut National de la Recherche Scientifique, Montreal, QC, Canada, in 2019. From 2011 to 2015, he was with the Sahand University of Technology, Tabriz. He is currently a Horizon Postdoctoral Fellow with Concordia University, Montreal. He is the lead author of the book *Toward 6G: A New Era of Convergence* (Wiley-IEEE Press, 2021). His research interests include 6G networks, tactile internet, FiWi networks, and multiaccess edge computing. He was awarded the 2019–2020 Best Doctoral Thesis Prize of INRS for his research on the tactile internet. He was a recipient of the Doctoral Research Scholarship from the B2X Program of Fonds de Recherche du Quebec-Nature et Technologies.



ROCH H. GLITHO (Life Senior Member, IEEE) received the M.Sc. degree in business economics from the University of Grenoble, France, the M.Sc. degree in pure mathematics and the M.Sc. degree in computer science from the University of Geneva, Switzerland, and the Ph.D. (Tech.Dr.) degree in informatics from the Royal Institute of Technology, Stockholm, Sweden. He is currently a Full Professor with Concordia University, where he holds the Canada Research Chair. He also holds the Ericsson/ENCQOR-5G Senior Industrial Research Chair of Cloud and Edge Computing for 5G and Beyond. In addition, he is a Professor extraordinaire with the Computer Science Department, University of the Western Cape, Cape Town, South Africa. He has worked in industry and has held several senior technical positions (e.g., a Senior Specialist, the Principal Engineer, and an Expert) with Ericsson Research, Sweden and Canada. He served as an IEEE Distinguished Lecturer and the Editor-in-Chief for *IEEE Communications Magazine* and the IEEE COMMUNICATIONS SURVEYS AND TUTORIALS.



JOHAN EKER received the Ph.D. degree in automatic control from Lund University, in 1999. He joined the Ptolemy Group, UC Berkeley. Since 2023, he has been a Professor of automatic control with Lund University, with a focus on real-time systems. He is currently a Principal Researcher with Ericsson Research, Sweden. He is the Head of the WASP Research Arena on Data-Driven Operations (WARA-Ops). He is the Co-Designer of the CAL Actor Language, which is part of the MPEG Standard ISO/IEC 23001-4:2011. He holds over 70 granted patents in the areas of telecom, the IoT, and cloud computing. He is participating in a range of program committees and research projects on topics, such as real-systems, signal processing, software development, cloud technology, brain–computer interfaces, and AI. He is involved in the operation of the Ericsson Research Data Center and works with industrial cloud applications. His current research interests include control of large-scale compute systems, but his research interests range from programming language design for parallel hardware, real-time control systems, mobile communications, software design for mobile devices, adaptive resource management, the IoT, and cloud technology.



RAQUEL A. F. MINI received the B.Sc., M.Sc., and Ph.D. degrees in computer science from the Federal University of Minas Gerais (UFMG), Brazil. She is currently a Researcher with Ericsson Research, involved in the cloud systems and platforms area. Before joining Ericsson Research, in 2021, she worked for more than 20 years as a Professor of computer science in Brazil. Her research interests include sensor networks, the IoT, ubiquitous computing, and cloud computing.